1. Complexity Table / Fast Lookup

Algorithm/Data Struct	Time	Space	Notes
Hash search/insert/delete	expected $O(1)$, worst $O(n)$	$O(n)$	α=n/m
BST ops	$O(h)$	$O(n)$	h $\log n$ balanced, n worst
Inorder BST	$\Theta(n)$	$O(h)$	sorted output
Insertion sort	best $\Theta(n)$, worst $\Theta(n^2)$	$O(1)$	stable
Merge sort	$\Theta(n \log n)$	$\Theta(n)$	stable
Quicksort randomized	expected $\Theta(n \log n)$	$O(\log n)$	worst $\Theta(n^2)$
Counting sort	$\Theta(n+k)$	$\Theta(n+k)$	integer keys
Heapify	$O(\log n)$	recursion	assumes children heaps
Build heap	$\Theta(n)$	$O(1)$	bottom-up
Heapsort	$\Theta(n \log n)$	$O(1)$	not stable
PQ insert/extract	$O(\log n)$	$O(n)$	heap
BFS	$\Theta(V+E)$	$\Theta(V)$	unweighted shortest paths
DFS	$\Theta(V+E)$	$\Theta(V)$	topo/cycles/SCC
Topological sort	$\Theta(V+E)$	$\Theta(V)$	DAG only
Kosaraju SCC	$\Theta(V+E)$	$\Theta(V+E)$	2 DFS
Union-Find op	$O(\alpha(n))$	$O(n)$	rank+compression
Kruskal	$O(E \log E)$	$O(V)$	sort + DSU
Prim	$O(E \log V)$	$O(V+E)$	PQ
Dijkstra	$O(E \log V)$	$O(V+E)$	no negative edges
Bellman-Ford	$\Theta(VE)$	$O(V)$	negative edges OK
DAG shortest path	$\Theta(V+E)$	$O(V)$	topo order
Ford-Fulkerson	$O(E\lvert f^*\rvert)$	$O(V^2+E)$	integer caps, residual
Edmonds-Karp	$O(VE^2)$	$O(V^2+E)$	BFS residual
Rod cutting	$\Theta(n^2)$	$\Theta(n)$	DP
Matrix chain	$\Theta(n^3)$	$\Theta(n^2)$	DP
LCS	$\Theta(nm)$	$\Theta(nm)$	DP
Knapsack 0/1	$\Theta(nW)$	$\Theta(nW)/\Theta(W)$	pseudo-poly
Coin change	$\Theta(W \cdot \#\text{coins})$	$\Theta(W)$	unbounded

Exam Pattern Matcher

new optimization + choices	DP: state/base/transition/order/answer/proof
local best + exchange	greedy: safe choice + induction
connect all min weight	MST: Kruskal/Prim + cut property
unweighted shortest path	BFS levels
weighted no negative	Dijkstra
negative weights / cycle	Bellman--Ford
dependencies / precedence	DAG + topological order
capacity / matching / disjoint paths	max-flow reduction
expected count	indicators + linearity
online unknown future	competitive ratio vs OPT
min max feasible value	binary search answer + monotone feasible
mutual reachability forbidden	SCC condensation / comp[u]=comp[v]
capacitated assignment	max-flow with integral capacities
tree path/subtree optimum	postorder DP: inside child / through root
tree traversals reconstruct	inorder+post/pre split at root
time-dependent edges	earliest-arrival Dijkstra/events
grid/assignment/paths	bipartite matching / max-flow

Exam Writeup Checklist

Algorithm: input/output, data structs, return value. Correctness: complete invariant/induction/exchange proof, not idea. DP: State, Base, Transition, Order, Answer, Runtime, Space. Bounds: define n, V, E, W; justify loops. Conventions: $<$ vs $\leq$, inclusive/exclusive, 0/1-index, reachable vs global negative cycle.

Proof Bank: Generic Templates

Loop inv. Init true before loop. Maint: one step preserves. Term: invariant + stop condition $\Rightarrow$ answer.

Induction/substitution. Claim upper/lower bound with constants. State IH for all smaller inputs. Substitute IH. Collect terms. Pick constants so inequality holds. Base cases: choose c large enough for all $n < n_0$.

Substitution trap. If $T(n) \leq c\sum_i (a_i n)^p + bn$ and $\sum_i a_i^p = 1$, then $cn^p + bn$ FAILS. No slack absorbs $+bn$. Strengthen: $T(n) \leq cn^p - dn^q$ with lower-order subtraction.

Midterm recurrence. $T(n) = T(4n/5) + T(3n/5) + an$. Guess $\Theta(n^2)$ because $(4/5)^2 + (3/5)^2 = 1$. Lower: $T(n) \geq T(4n/5)+T(3n/5) \geq c((4n/5)^2 + (3n/5)^2) = cn^2$. Upper must be strengthened: assume $T(m) \leq cm^2 - dm$ for all $m < n$. Then $T(n) \leq c(4n/5)^2 - d(4n/5) + c(3n/5)^2 - d(3n/5) + an = cn^2 - 7dn/5 + an \leq cn^2 - dn$ iff $d \geq 5a/2$. Choose d so, then choose c large for base cases.

D&C. Induct on size. Recursive calls solve smaller problems. Combine covers all optimum cases and constructs/chooses correct result.

DP. State meaning. Base correct. Induct in fill order. Transition enumerates all valid first/last choices; dependencies already optimal. Optimal solution uses one enumerated choice; every candidate feasible. Therefore $dp[s]$ optimum. Write answer $= dp[...]$.

Parametric search. Guess answer M. Build feasible(M). Prove monotone: if M works then any larger M works (minimization). Binary search smallest feasible M. Correctness = binary search invariant + feasible predicate exact.

Greedy. Safe choice: for optimum O containing current partial solution, if chosen $e \notin O$, exchange $f \in O$ with e without worsening. Then induct.

Proof Bank: Sorting / Hash / DS

BST inorder. Induct on subtree. Left keys $<$ root, right keys $\geq$ root. IH sorts left/right. Left,root,right sorted and complete.

Rank/interval. Store $size = 1 + size(L) + size(R)$. Rank follows one search path; when going right add $size(L)+1$. Thus $O(h)$. $[a,b] = rank_{\leq}(b) - rank_{<}(a)$.

Heapify. Children are heaps. Swap with larger child if violated; parent fixed. Only swapped child may violate; recurse to leaf. Cost height $O(\log n)$.

BuildHeap. Leaves are heaps. Bottom-up: children already heaps, so Heapify fixes subtree. Runtime: nodes height $h \leq n/2^{h+1}$, total $\sum_h O(h)n/2\hat{}h = O(n)$.

Sorting lower bound. Comparison sort decision tree needs $\geq n!$ leaves. Worst-case comparisons $\geq$ height $\geq \log_2(n!) = \Omega(n \log n)$.

Quicksort exp. Pair compared at most once. $X_{ij} = 1$ if ranks i, j compared. Compare iff first pivot among ranks $i..j$ is i or j, so $P(X_{ij}) = 2/(j-i+1)$. Sum $O(n \log n)$.

Hashing. Single key: for key x, $X_y = 1[h(y) = h(x)]$. Universal hashing gives $E[X_y] \leq 1/m$, so expected chain collisions $\leq (n-1)/m = \Theta(\alpha)$ and search $O(1+\alpha)$. Total collisions: $X = \sum_{i<j} X_{ij}$, $X_{ij} = 1[h(x_i) = h(x_j)]$, hence $E[X] \leq \binom{n}{2}/m = n(n-1)/(2m) = \Theta(n2/m)$. k-collision: $E[\#k\text{-collisions}] \leq \binom{n}{k}/m^{k-1}$. Markov: $P[X \geq 1] \leq E[X]$.

Hash mini. Length-ℓ keys: pair collision prob $1/m^\ell$, so $E[\#collisions] = \binom{n}{2}/m^\ell$; if $m^\ell \geq \binom{n}{2}$ then $P[\exists collision] \leq 1$. Random q-color vertices: edge mono prob $1/q$, so $E[\#mono] = |E|/q$.

DSU. Union by rank attaches smaller rank under larger. Rank increases only on equal-rank merge. Path compression preserves reps and shortens future paths. With both: m ops in $O(m\alpha(n))$.

Proof Bank: Graph Traversal

BFS. Queue processed in nondecreasing distance. Discover v from u: $d[v] = d[u] + 1$. Shorter path would come from smaller-distance vertex processed earlier, so v would already be discovered.

DFS/topo. Directed cycle iff DFS has back edge to GRAY ancestor. DAG has no back edges. For every edge $u \to v$, $f[u] > f[v]$; decreasing finish time is topo order.

DFS edge facts. Parenthesis theorem: finish intervals disjoint or nested; nested iff descendant. White-path theorem: v descendant of u iff when u discovered, there is a white path $u \rightsquigarrow v$.

SCC/Kosaraju. Condensation graph is DAG. First DFS finish order puts source SCCs late. In reversed graph, DFS from highest finish cannot leave its SCC to an unvisited SCC. Each second-pass DFS outputs one SCC.

SCC reachability pairs. $u \rightsquigarrow v$ and $v \rightsquigarrow u$ iff $comp[u] = comp[v]$. For forbidden pair (u, v) reject iff same component. Correct because SCC is maximal mutual reachability. Runtime $O(V+E)$.

DAG SP/LP. Topo order processes every predecessor before vertex. When processing u, $dist[u]$ already final over all paths ending at u; relaxing outgoing edges propagates optimum once. Runtime $\Theta(V+E)$.

Proof Bank: MST / Shortest Paths / Flow

Cut property. Let e be light edge crossing cut respecting A. Take MST $T \supseteq A$ not using e. Add e creates cycle with another crossing edge f. Since $w(e) \leq w(f)$, $T - f + e$ is MST containing $A \cup \{e\}$.

Cycle property. If MST T contains unique heaviest edge e on cycle, remove e; cycle has another edge f reconnecting cut with $w(f) < w(e)$. Then $T - e + f$ cheaper, contradiction.

Kruskal. DSU components define cut respecting chosen edges. Next accepted edge is lightest crossing some component cut. Cut property says safe. Induct until $|V| - 1$ edges.

Prim. $S =$ vertices in tree. Heap chooses lightest edge crossing $(S, V \setminus S)$. Cut property says safe. Add one vertex; repeat.

Dijkstra. $S =$ finalized. Pick outside x with min tentative dist. Any alternative path first crosses from S to outside at (u, v). Since $w \geq 0$, path length $\geq d[u] + w(u,v) \geq d[v] \geq d[x]$. So $d[x]$ final.

Bellman-Ford. After i passes, all shortest paths using $\leq i$ edges are correct. Induct by last edge relaxation. If no negative cycle, shortest simple path has $\leq V - 1$ edges. If pass V improves, reachable negative cycle.

Max-flow. If no augmenting path exists, let S be vertices reachable from s in residual graph. For every edge $u \in S, v \notin S$: $cf(u,v) = 0$, therefore $f(u,v) = c(u,v)$. For every edge $u \notin S, v \in S$: $cf(v,u) = 0$, therefore $f(u,v) = 0$.

Hence net flow crossing cut equals cut capacity: $|f| = c(S,T)$.

Since every flow $\leq$ every cut, flow and cut are both optimal.

Flow cert. Any flow value $\leq$ any cut capacity. Flow value F + cut capacity F proves both optimal.

Bipartite matching flow. Caps 1 force each left/right vertex used at most once. Integral capacities give integral max flow. Unit flow through $l \to r$ is matched pair; every matching gives same-size flow.

Cap assignment flow. Build $s \to$ people cap 1, person $\to$ acceptable item cap 1, item $\to t$ cap capacity. Feasible iff maxflow = #people. Integral flow = assignment; assignment = integral flow.

SP transform. Multiplying every edge weight by $a > 0$ preserves all shortest paths. Adding a to every edge does not: paths with different number of edges get different extra cost.

Resource SP. State graph (v, b) where $b =$ used budget / remaining battery. Add transition only if resource feasible. Run BFS/Dijkstra depending on edge costs. Size $O(VB + EB)$. Correct since every feasible walk corresponds to one state path and vice versa.

2D sorted matrix. Rows/cols sorted. Start top-right. If $A[i,j] > x$, move left; if $A[i,j] < x$, move down. Each step deletes one impossible row/col, so $O(n+m), O(1)$.

Proof Bank: Trees / Exam Reductions

Tree DP. Postorder. Return height/best info from children. Best at u is max(best in left, best in right, best crossing/using u). Induct on subtree: any optimal object either lies fully in one child or uses u.

Tree diameter. Let $h(u) = 1 + \max(h_L, h_R)$, empty height -1. $diam(u) = \max(diam_L, diam_R, h_L + h_R + 2)$. Path either inside one child or crosses through u.

Root path. Longest simple path forced to pass through root: $h(root.left) + h(root.right) + 2$ with empty height -1. Compute heights postorder in $O(n)$.

Tree rebuild. Inorder+postorder: root=last(post); split inorder at root; subtree sizes split postorder; recurse. Preorder+inorder: root=first(pre). Pre+post alone not unique.

Unicyclic spanning trees. Connected graph with $|E| = |V|$ has exactly one cycle C. Every spanning tree = delete exactly one edge of C. Deleting non-cycle edge disconnects; deleting cycle edge breaks only cycle. Hence #spanning trees $= |C|$.

Proof Bank: Online / Random

LRU paging. Cache size k. Phase has $\leq k$ distinct pages; new phase begins at request of $(k+1)$-st distinct. LRU: $\leq k$ misses/phase, because requested page survives until k other distinct pages requested. OPT: ≥ 1 miss/completed phase, because phase + first request of next phase contains $k+1$ distinct pages but cache holds k. Thus LRU $\leq k \cdot \text{OPT} + O(k)$; LRU is k-competitive.

Verbose Proof Micro-Stencils

Correctness opening: Start with a precise claim: after the algorithm terminates, the returned object is feasible and has objective value no worse than any feasible object. Then prove feasibility and optimality separately.

Induction base habit: Say exactly which subproblems are smallest and why their table/return value is correct. Do not write ``trivial'' unless the value is actually shown, e.g. dp[0]=0 or empty subtree height=-1.

Induction step habit: Assume all strict subproblems used by the transition are correct. Show every transition candidate is feasible, and the optimum's first/last choice appears among the candidates.

Exchange proof form: Let O be an optimum agreeing with greedy for as long as possible. If greedy choice g differs from O's next choice o, replace o by g; prove feasibility remains and weight/count is not worse. Contradiction to maximal agreement.

Greedy lower-bound proof: To prove greedy is optimal, also prove a matching lower bound: any solution needs at least X objects/cost. Then show greedy achieves X. Common in interval partitioning and stabbing.

Cut proof exactness: For MST safe edge proofs, the cut must respect already chosen edges A: no edge in A crosses it. Then swapping along the cycle cannot remove an edge of A.

Cycle proof exactness: For MST cycle proofs, removing the heavy edge disconnects T into a cut. The replacement edge on the cycle crosses the same cut, reconnects T, and has smaller/equal weight.

Reduction correctness: Prove both directions: any original solution maps to a feasible constructed solution of same value, and any constructed solution maps back to a valid original solution of same value. Then optimum values equal.

Flow reduction proof: Integral capacities imply an integral max flow. Unit flow through a gadget means the corresponding discrete choice is selected; capacity constraints exactly encode usage limits.

Matching proof: Flow->matching: cap 1 on s->L and R->t gives at most one matched edge per vertex. Matching->flow: send one unit along s-l-r-t for each matched pair.

Min-cut certificate: After max-flow, BFS in residual graph from s gives S. Edges S->T are saturated and T->S carry zero flow, so value crossing cut equals cut capacity.

Shortest path reduction: When adding state dimensions, prove bijection-ish: every feasible original walk corresponds to a path in the state graph, and every state path decodes to a feasible walk.

Dijkstra finality: The key sentence: when x is extracted, any alternative path to x must first leave the finalized set by some edge (u,v); nonnegative weights make that path at least dist[v] >= dist[x].

Bellman-Ford invariant: After i full passes, dist[v] is at most the length of the shortest s->v path using at most i edges, and no smaller value than any such path is needed. Last-edge relaxation proves it.

Negative-cycle proof: If pass V still relaxes an edge reachable from s, the improving walk has V edges, so some vertex repeats; the repeated cycle must have negative total weight.

BFS shortest proof: Use contradiction: let v be first vertex discovered with nonshortest distance. Its predecessor on a shortest path has smaller distance and was processed earlier, so it would have discovered v correctly.

DFS cycle proof: Directed cycle -> during DFS, take the first vertex on the cycle discovered; following the cycle reaches a gray ancestor edge before it finishes. Gray edge -> ancestor path plus edge forms a cycle.

Topo proof: In a DAG DFS has no back edge. For every edge u->v, v finishes before u whether discovered from u or already black. Therefore decreasing finish time respects every edge.

SCC proof: Condensation is a DAG. The second DFS in reverse graph starts from a source of remaining reverse condensation, equivalently a sink/source by finish order, so it cannot escape its SCC.

Binary search invariant: Maintain answer in [l,r]. Mid test discards only values impossible by monotonicity. Termination l=r gives the unique smallest feasible or largest feasible depending on direction.

Feasibility proof: For parametric search, prove feasible(M) is exact, not just sufficient. Usually: greedy uses the minimum possible resources for this M, so if greedy fails no valid solution exists.

Divide-farm proof: For fixed M, greedy makes each segment as long as possible. After t segments greedy covers at least as much prefix as any t-segment valid partition; hence greedy minimizes number of segments.

Interval scheduling proof: Let g be earliest finishing interval. For any optimum, replace its first interval by g. Since g finishes no later, all later intervals remain compatible; size unchanged. Induct on suffix.

Weighted interval DP proof: Optimal solution for first i intervals either skips i or takes i. If it takes i, all remaining chosen intervals lie among first p(i). These two cases are exhaustive and disjoint.

Travel-pass DP proof: In an optimum covering first uncovered day d[i], the bought pass has one of the allowed lengths L and covers exactly until d[i]+L-1. The remaining subproblem starts at next(i,L).

Knapsack proof: For item i, optimum either excludes it, giving dp[i-1][w], or includes it, forcing capacity w-weight[i] for previous items. No third possibility exists.

LCS proof: For prefixes i,j: if last chars match, some optimum can use them and reduce to i-1,j-1. If they differ, at least one last char is not used, giving max of the two smaller prefixes.

Matrix-chain proof: The final multiplication splits the product at some k. Costs before that split are independent subchains, so optimality follows by trying every k and using optimal subsolutions.

Rod-cutting proof: Look at the leftmost piece length i in an optimal cutting. The remainder length n-i must itself be cut optimally, else replace it and improve total revenue.

Optimal BST proof: Choosing root r increases depth of every key in left/right subtrees by 1, adding total interval probability w[i,j]. Subtrees are independent optimal BST subproblems.

Tree DP proof: Any optimal object in subtree u either lies completely in one child subtree or uses u / crosses between children. The recurrence takes the max over exactly these cases.

Tree reconstruction proof: Root position in inorder uniquely separates left and right key sets. Subtree sizes then uniquely split preorder/postorder blocks, so recursion reconstructs the only tree.

Heapify proof: Before heapify, both child subtrees are heaps. Swapping with the larger child fixes the parent; the only possible violation moves down one level. Eventually a leaf is reached.

Build-heap proof: Process internal nodes from bottom to top. When heapify(i) runs, both children already root heaps, so it makes subtree i a heap. At the root the whole array is a heap.

Insertion sort proof: Before iteration j, prefix A[0..j) is sorted and contains the original prefix elements. Inner loop shifts larger elements right; inserting key restores sorted prefix A[0..j+1).

Merge proof: Loop invariant: output contains the smallest consumed elements from both sorted halves, in sorted order. Taking the smaller head preserves sortedness; copying leftovers completes all elements.

Quicksort proof: Partition puts pivot in final rank: all left <= pivot, all right > pivot. Recursive calls sort the two sides; concatenating left,pivot,right is sorted and complete.

Counting-sort proof: Prefix count C[x] equals number of input keys <=x. Scanning input right-to-left places each key at the last free position of its value, giving sorted stable output.

Hash expectation proof: Define one indicator per bad pair/event. Compute probability for a fixed pair, then sum expectations. Independence between different pairs is not needed for the expectation.

Quicksort expectation proof: Elements of ranks i,j are compared iff the first pivot chosen among ranks i..j is one of them. That probability is 2/(j-i+1); summing pairs gives O(n log n).

Lower-bound proof: For comparison sorting, leaves of decision tree must distinguish all n! input orders. A binary tree of height h has at most 2^h leaves, so h>=log(n!).

Online upper proof: Partition the request sequence into phases. Prove ALG cost per phase <=A and OPT cost per phase >=B, then sum phases and absorb boundary phases into constant c.

Online lower proof: Adversary observes the deterministic algorithm's action and chooses when to stop/continue. Compare on the same final sequence against offline OPT.

Ski-rental proof: If T<=B, rent-then-buy pays T=OPT. If T>B, it pays at most B rent plus B buy, while OPT pays B; ratio <=2.

LRU phase proof: In one phase there are at most k distinct pages, so after a page is requested, LRU will not evict it before k other distinct pages appear, which would start a new phase.

Experts potential proof: Track total weight W. Algorithm mistake means wrong side has at least half W, so W shrinks by factor <=(1+beta)/2. Best expert gives lower bound beta^m*.

Prophet threshold proof: Separate value into threshold part tau*P[accept] and overshoot part. If all earlier variables miss threshold, each positive overshoot has chance to be captured; combine with tau=z/2.

Secretary proof: Condition on best arriving at position i>r. Success iff best among first i-1 positions is in sample. Probability sum over i: (1/n)*(r/(i-1)).

Runtime proof: Count objects actually processed: each vertex enqueued once, each edge scanned once, each DP state filled once, each heap operation O(log size). State dimensions after reductions.

Off-by-one proof: State intervals explicitly. Half-open [l,r) usually makes size r-l, empty when l=r, and split [l,m),[m,r). Closed [i,j] needs base i>j or i=j.

Final answer habit: After proof, write the exact returned value: dp[n], dp[1], dist[t], maxflow value, selected edges, or reconstructed choices. Missing this often loses points.

2. Complexity / Asymptotics

$\log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n! \ll n^n$

$O(g)$: $f \leq cg$ eventually. $\Omega(g)$: $f \geq cg$. $\Theta(g) = O \cap \Omega$. $o(g)$: $f/g \to 0$. $\omega(g)$: $f/g \to \infty$. Stirling: $n! \sim \sqrt{2\pi n}(n/e)^n$, so $\log(n!) = \Theta(n \log n)$.

Useful sums:

$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$, $\sum_{i=0}^{\log n} n = n \log n$, $\sum_{i\geq 0} 2^{-i} = 2$, $\sum_{h\geq 0} h2^{-h} = 2$

Master / recurrence solver:

Master: $T(n) = aT(n/b) + f(n)$. Let $d = \log_b(a)$. Compare $f(n)$ with n^d.

f smaller: $f = O(n^{d-\varepsilon})$ $T = \Theta(n^d)$

same: $f = \Theta(n^d \log^k n)$ $T = \Theta(n^d \log^{k+1} n)$

f bigger: $f = \Omega(n^{d+\varepsilon})$ $T = \Theta(f)$ if regularity

Fast mental version if $f = n^k$:

$k < d$	leaves win	$\Theta(n^d)$
$k = d$	tie/layers equal	$\Theta(n^d \log n)$
$k > d$	root/work wins	$\Theta(n^k)$

Non-equal splits: $T = \sum_i a_i T(b_i n) + \Theta(n^k)$. Find p s.t. $\sum_i a_i b_i^p = 1$.

$p > k$	leaves win	$\Theta(n^p)$
$p = k$	tie	$\Theta(n^p \log n)$
$p < k$	root/work wins	$\Theta(n^k)$

Trap: if $\sum_i a_i b_i^p = 1$, plain cn^p proof has no slack; use $cn^p - dn^q$.

Akra--Bazzi mini: $T(n) = \alpha T(an) + \beta T(bn) + \Theta(n^d)$. Find p: $\alpha a^p + \beta b^p = 1$. Then $p > d \Rightarrow \Theta(n^p)$, $p = d \Rightarrow \Theta(n^d \log n)$, $p < d \Rightarrow \Theta(n^d)$. Recurrences / runtimes:

```
BinSearch: T(n/2)+1 -> log n
Merge/MaxSub: 2T(n/2)+n -> n log n
Naive MM: 8T(n/2)+n^2 -> n^3
Strassen: 7T(n/2)+n^2 -> n^log2(7)
Heapify: T(2n/3)+1 -> log n
QSort(w): T(n-1)+n -> n^2
QSort(b): 2T(n/2)+n -> n log n
BF: (V-1)E relax -> VE
BFS/DFS: V+E

T(n/2)+1->log n     T(n/2)+n->n
2T(n/2)+1->n        2T(n/2)+n->n log n
3T(n/2)+n->n^log2(3)
7T(n/2)+n^2->Strassen
8T(n/2)+n^2->n^3
T(n-1)+n->n^2
```

3. Basic Data Structures

Stack / Queue / Linked List

Stack	push/pop/top	$O(1)$, array end, underflow if empty
Queue	enqueue/dequeue	$O(1)$ with circular array / deque
Linked list	search/insert/delete	search $O(n)$, known-node delete $O(1)$

Doubly linked list insert head: `x.next=head; if head!=NIL head.prev=x; head=x; x.prev=NIL`. Delete known node: reconnect `prev.next` and `next.prev`; finding node still $O(n)$.

Hash Tables

Idea: bucket $h(key) \bmod m$. Collisions: chaining / probing. Load factor $\alpha = n/m$.

Hash ops: insert/delete expected O(1), search expected O(1+alpha), all worst O(n); m=Theta(n) => alpha=Theta(1). Expected total collisions: $X = \sum_{i<j} X_{ij}$, where $X_{ij} = 1[h(x_i) = h(x_j)]$. Universal hashing gives P[Xij=1]<=1/m, so E[X]=sum E[Xij] <= C(n,2)/m = n(n-1)/(2m)=Theta(n2/m). For one key: $\mathbb{E}[\#\text{collisions}] \leq (n-1)/m = \Theta(\alpha)$, hence expected search $O(1+\alpha)$. Traps: HashMap/HashSet $O(1)$ is expected/amortized, not worst-case. Bad hashing / all collisions $\Rightarrow O(n)$.

k-collision stencil: $X_S = 1$ if fixed k-set collides. $P[X_S = 1] \leq 1/m^{k-1}$. $E[X] = \sum_{|S|=k} E[X_S] \leq \binom{n}{k}/m^{k-1}$. If need existence prob: $P[X \geq 1] \leq E[X]$.

Binary Search Trees

Binary search (sorted array):

```
fn binary_search(a: Vec<_>,x){
  l=0; r=a.len();
  while l<r {
    m=(l+r)/2;
    if a[m]==x { return Some(m); }
    if a[m]<x  { l=m+1; }
    else       { r=m; }
  }//lower_bound = first >= x:
  None//First position where the value is >= x.
} //upper_bound = first > x:
//First position where the value is > x.
```

BST property: BST property: all keys in L(x) < x.key; all keys in R(x) >= x.key.

BST ops search/insert/delete/min/max/succ/pred O(h); inorder Theta(n); balanced h=Theta(log n), degenerate h=Theta(n).

```
search(x,k):
  while x != NIL and x.key != k:
    if k < x.key: x = x.left
    else:         x = x.right
  return x

min(x): while x.left  != NIL: x = x.left; ret x
max(x): while x.right != NIL: x = x.right;ret x

successor(x):
  if x.right != NIL: return min(x.right)
  y = x.parent
  while y != NIL and x == y.right:
    x = y
    y = y.parent
  return y
insert(T,z):
  y=NIL; x=T.root
  while x!=NIL:
    y=x
    if z.key<x.key: x=x.left else: x=x.right
  z.parent=y
  if y==NIL: T.root=z
  else if z.key<y.key: y.left=z
  else: y.right=z
transplant(T,u,v):
  if u.parent==NIL: T.root=v
  else if u==u.parent.left: u.parent.left=v
  else: u.parent.right=v
  if v!=NIL: v.parent=u.parent

delete(T,z):
  if z.left==NIL: transplant(T,z,z.right)
  else if z.right==NIL: transplant(T,z,z.left)
  else:
    y=min(z.right)
    if y.parent!=z:
      transplant(T,y,y.right);
      y.right=z.right; y.right.parent=y
    transplant(T,z,y);
    y.left=z.left; y.left.parent=y
```

Delete cases: no left, no right, or replace by successor. All $O(h)$.

Traversals:

```
inorder(x):  if x!=NIL: inorder(x.left); visit(x); inorder(x.right)
preorder(x): if x!=NIL: visit(x); preorder(x.left); preorder(x.right)
postorder(x):if x!=NIL: postorder(x.left); postorder(x.right); visit(x)
```

inorder(BST)=sorted order.

Augmented BST / rank queries: sz=1+L+R, rank O(h). [a,b]=r<=b-r<a, (a,b]=r<=b-r<=a, [a,b)=r<b-r<a, (a,b)=r<b-r<=a.

4. Sorting Algorithms

Insertion Sort

In-place, stable. Invariant: before iteration j, prefix $A[0..j)$ sorted.

```
fn insertion_sort(a: &mut Vec<i32>) {
  for j in 1..a.len() {
    let key = a[j];
    let mut i = j;
    while i > 0 && a[i-1] > key {
      a[i] = a[i-1]; i -= 1;
    }
    a[i] = key;
  } // Best Theta(n), worst/avg Theta(n^2), space O(1)
}
```

Merge Sort

Stable, simple version uses Theta(n) temp.

```
fn merge_sort(a,l,r){
  if r-l <= 1 { return; }
  let m=(l+r)/2;
  merge_sort(a,l,m);
  merge_sort(a,m,r);
  merge(a,l,m,r);
}
fn merge(a,l,m,r){
  let (mut i,mut j)=(l,m); let mut t=Vec::new();
  while i<m && j<r {
    if a[i] <= a[j] { t.push(a[i]); i+=1; }
    else { t.push(a[j]); j+=1; }
  }
  while i<m { t.push(a[i]); i+=1; }
  while j<r { t.push(a[j]); j+=1; }
  for k in 0..t.len(){ a[l+k]=t[k]; }
}
```

T(n)=2T(n/2) + Theta(n) = Theta(n log n)

Quicksort

In-place. Partition invariant: A[l..i]<= p, A[i+1..j]>p, unknown after j, pivot at r.

```
fn partition(a,l,r)->usize{
  let p=a[r]; let mut i=l;
  for j in l..r {
    if a[j] <= p { a.swap(i,j); i+=1; }
  }
  a.swap(i,r); i
}
fn quicksort(a,l,r){
  if l>=r { return; }
  let q=partition(a,l,r);
  if q>0 { quicksort(a,l,q-1); }
  quicksort(a,q+1,r);
}
```

Balanced/expected randomized Theta(n log n), worst Theta(n^2), stack expected O(log n) worst O(n).

Counting Sort / Linear Sorting

Integer keys $0..k$, stable with prefix sums, not comparison-based.

```
fn counting_sort(a:&Vec<usize>, k:usize)->Vec<usize>{
  let mut c=vec![0;k+1]; let mut out=vec![0;a.len()];
  for &x in a { c[x]+=1; }
  for i in 1..=k { c[i]+=c[i-1]; }
  for idx in (0..a.len()).rev(){
    let x=a[idx]; c[x]-=1; out[c[x]]=x;
  }
  out
}
```

$\Theta(n+k)$ time/space. Stable with prefix sums. Needs bounded integer keys.

5. Heaps / Priority Queues / Heapsort

Max-Heapify

Assume children subtrees are heaps; only $A[i]$ may violate. 0-index:

```
fn max_heapify(a:&mut Vec<i32>, i:usize, heap_size:usize){
  let (l,r)=(2*i,2*i+1); let mut largest=i;
  if l<heap_size && a[l]>a[largest] { largest=l; }
  if r<heap_size && a[r]>a[largest] { largest=r; }
  if largest!=i { a.swap(i,largest); max_heapify(a,largest,heap_size); }
}
```

$O(height(i)) \leq O(\log n)$. Trap: during heapsort, heap_size $\neq$ a.len().

Build Max Heap

```
fn build_max_heap(a:&mut Vec<i32>){
  let n=a.len();
  for i in (0..(n/2)).rev(){ max_heapify(a,i,n); }
}
```

Tight: nodes height $h \leq n/2^{h+1}$, cost $O(h)$, so $\sum_h nh/2\hat{}h = O(n)$; hence BuildHeap $\Theta(n)$, not $\Theta(n \log n)$.

Heapsort

```
fn heapsort(a:&mut Vec<i32>){
  build_max_heap(a); let mut h=a.len();
  while h>1 { a.swap(0,h-1); h-=1; max_heapify(a,0,h); }
}
```

Build $\Theta(n)$ + n extracts $O(\log n)$ = $\Theta(n \log n)$, space $O(1)$, not stable.

Priority Queue

```
fn heap_extract_max(a:&mut Vec<i32>)->Option<i32>{
  if a.is_empty(){return None;}
  let ans=a[0]; let last=a.pop().unwrap();
  if !a.is_empty(){ a[0]=last; max_heapify(a,0,a.len()); }
  Some(ans)
}
fn heap_increase_key(a:&mut Vec<i32>, mut i:usize, key:i32){
  if key<a[i]{return;} a[i]=key;
  while i>0 { let p=(i-1)/2; if a[p]>=a[i]{break;} a.swap(i,p); i=p; }
}
fn max_heap_insert(a:&mut Vec<i32>, key:i32){
  a.push(i32::MIN); heap_increase_key(a,a.len()-1,key);
}//Ops insert/extract/increase-key O(log n)
```

6. Divide & Conquer Algorithms

Karatsuba Multiplication

$$x = x_h B^m + x_l, \quad y = y_h B^m + y_l$$
$$a = x_h y_h, \quad b = x_l y_l, \quad c = (x_h + x_l)(y_h + y_l)$$
$$xy = aB^{2m} + (c - a - b)B^m + b$$

3 recursive multiplications.

3 recursive multiplications: $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Maximum Sum Subarray

Problem:
Input A[0..n) Output max sum contiguous nonempty subarray
D&C idea:
best subarray is:

- entirely left
- entirely right
- crosses middle

```
fn max_crossing(a,l,m,r){
  // left suffix [l,m)
  left_sum=-INF;
  sum=0;
  for i in (l..m).rev(){
    sum+=a[i];
    left_sum=max(left_sum,sum);
  }
  // right prefix [m,r)
  right_sum=-INF;
  sum=0;
  for j in m..r {
    sum+=a[j];
    right_sum=max(right_sum,sum);
  }
  left_sum+right_sum
}
fn max_subarray(a,l,r){
  if r-l==1 {return a[l]}
  m=(l+r)/2;
  left=max_subarray(a,l,m);
  right=max_subarray(a,m,r);
  cross=max_crossing(a,l,m,r);
  max(left,max(right,cross))
}
```

Complexity: T(n)=2T(n/2)+Theta(n)=Theta(nlog n)

Linear Kadane:

```
fn kadane(a: &Vec<i32>) -> i32 {
    let mut best: i32 = a[0];
    let mut cur: i32 = a[0];
    for i in 1..a.len() {
        cur = a[i].max(cur + a[i]);
        best = best.max(cur);
    }
    best
}
```

Kadane invariant: cur[i] = max(A[i], cur[i-1]+A[i]), best[i]=max(best[i-1],cur[i])

Matrix Multiplication

Naive:

```
for i in 0..n {for j in 0..n {for k in 0..n {c[i][j] += a[i][k] * b[k][j];}}}
```

Complexity: Theta(n^3), Space: Theta(n^2)

D&C block formula (8 mults): C11=A11B11+A12B21, C12=A11B12+A12B22, C21=A21B11+A22B21, C22=A21B12+A22B22. Hence T(n)=8T(n/2)+Theta(n^2)=Theta(n^3).

Strassen (7 mults): M1=(A11+A22)(B11+B22), M2=(A21+A22)B11, M3=A11(B12-B22), M4=A22(B21-B11), M5=(A11+A12)B22, M6=(A21-A11)(B11+B12), M7=(A12-A22)(B21+B22). C11=M1+M4-M5+M7, C12=M3+M5, C21=M2+M4, C22=M1-M2+M3+M6.

```
fn strassen(A,B,n){
  if n==1 { return A*B }
  split A,B into A11..A22,B11..B22;
  M1..M7 = strassen(formulas above);
  C11=M1+M4-M5+M7; C12=M3+M5;
  C21=M2+M4;       C22=M1-M2+M3+M6;
  combine(C11,C12,C21,C22)
}
```

T(n)=7T(n/2)+Theta(n^2)=Theta(n^log2 7) $\approx$ Theta(n^2.807).

7. Dynamic Programming

Rod Cutting

revenue$[\ell]$ = best revenue length ℓ, first$[\ell]$ = first cut. Rod cutting: revenue[0]=0; revenue[l]=max over 1<=c<=l of price[c]+revenue[l-c].

```
fn rod_cut(price:Vec<i32>, n:usize){
  revenue=vec![0;n+1]; first=vec![0;n+1];
  for length in 1..=n {revenue[length]=-INF;
    for cut in 1..=length {
      candidate=price[cut]+revenue[length-cut];
      if candidate>revenue[length] {
        revenue[length]=candidate; first[length]=cut
      }
    }
  }
  (revenue,first)
}
```

Reconstruct: output `first[length]`, set `length-=first[length]`. Runtime $\Theta(n^2)$.

Matrix Chain Multiplication

Matrix i: $p_i \times p_{i+1}$. $dp[i][j]$ = min cost for $A_i \cdots A_j$. MCM recurrence: dp[i][i]=0; dp[i][j]=min over i<=k<j of dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1].

```
fn matrix_chain(p:Vec<usize>){     // optimal parenthesization
  n=p.len()-1; dp=vec![vec![0;n];n]; split=vec![vec![0;n];n];
  for len in 2..=n {for i in 0..=n-len {
    j=i+len-1; dp[i][j]=INF;
    for k in i..j {
      cost=dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1];
      if cost<dp[i][j] {dp[i][j]=cost; split[i][j]=k}
    }
  }}
  (dp,split)
}
```

Increasing chain length. $\Theta(n^2)$ states $\times \Theta(n)$ splits $= \Theta(n^3)$.

Optimal BST

Sorted keys K_i with successful probs p_i; optional unsuccessful q_i. $e[i][j] =$ min expected cost for subtree $i..j$, $w[i][j] =$ prob mass, $root[i][j] =$ best root. OBST: e[i][i-1]=0, w[i][i-1]=q[i-1]; e[i][j]=min over root r in [i,j] of e[i][r-1]+e[r+1][j]+w[i][j].

```
fn optimal_bst(p,q,n){
  for i in 1..=n+1 {e[i][i-1]=0; w[i][i-1]=q[i-1]}
  for len in 1..=n {for i in 1..=n-len+1 {
    j=i+len-1; e[i][j]=INF; w[i][j]=w[i][j-1]+p[j]+q[j];
    for r in i..=j {
      cost=e[i][r-1]+e[r+1][j]+w[i][j];
      if cost<e[i][j] {e[i][j]=cost; root[i][j]=r}
    }
  }}
  (e,root)
}
```

Runtime Theta(n^3), space Theta(n^2).

Longest Common Subsequence

$dp[i][j]$ = LCS length of $A[0..i)$, $B[0..j)$. LCS: dp[0][*]=dp[*][0]=0. If A[i-1]==B[j-1]: dp[i][j]=1+dp[i-1][j-1]; else max(up,left).

```
fn lcs(a:Vec<char>, b:Vec<char>){
  n=a.len(); m=b.len();
  dp=vec![vec![0;m+1];n+1];
  for i in 1..=n {for j in 1..=m {
    dp[i][j]=if a[i-1]==b[j-1]
      {1+dp[i-1][j-1]}
      else {dp[i-1][j].max(dp[i][j-1])}
  }}
  dp[n][m]
}
fn lcs_reconstruct(a,b,dp){
  i=a.len(); j=b.len(); ans=vec![];
  while i>0 && j>0 {
    if a[i-1]==b[j-1] {
      ans.push(a[i-1]); i-=1; j-=1
    }
    else if dp[i-1][j]>=dp[i][j-1] {i-=1}
    else {j-=1}
  }
  ans.reverse(); ans
}
```

$\Theta(nm)$ time, $\Theta(nm)$ or rolling $\Theta(m)$ space. Equal $\to$ diagonal+1; unequal $\to$ max(up,left).

Knapsack 0/1

$dp[i][c]$ = best value using first i items, capacity c. 0/1 knapsack: dp[i][c]=max(skip dp[i-1][c], take v[i-1]+dp[i-1][c-w[i-1]]) if w[i-1]<=c.

```
fn knapsack(w:Vec<usize>, v:Vec<i32>, cap:usize){ // choose item at most once
  n=w.len(); dp=vec![vec![0;cap+1];n+1];
  for i in 1..=n {for c in 0..=cap {
    dp[i][c]=dp[i-1][c];
    if w[i-1]<=c {
      dp[i][c]=dp[i][c].max(v[i-1]+dp[i-1][c-w[i-1]])
    }
  }}
  dp[n][cap]
}
// 1D 0/1: capacity descending. Unbounded: capacity ascending.
```

$\Theta(nW)$ pseudo-polynomial.

Coin Change / Change-Making

Coin change: mincoins[0]=0; mincoins[x]=min over coin<=x of 1+mincoins[x-coin].

```
fn coin_change(coins:Vec<usize>, target:usize){ // unlimited coins
  mincoins=vec![INF;target+1]; mincoins[0]=0;   // min number of coins
  for amount in 1..=target {for coin in coins {
    if coin<=amount {
      mincoins[amount]=mincoins[amount].min(1+mincoins[amount-coin])
    }
  }}
  mincoins[target]
}
```

Runtime $\Theta(target \cdot \#coins)$.

Longest Increasing Subsequence

Quadratic: LIS $O(n^2)$: $dp[i] = 1 + \max\{dp[j] : j < i, a[j] < a[i]\}$; answer=$\max dp$.

```
fn lis_n2(a:Vec<i32>){
  dp=vec![1;a.len()];
  for i in 0..a.len(){for j in 0..i{
    if a[j]<a[i]{dp[i]=dp[i].max(1+dp[j])}
  }}
  max(dp)
}
fn lis_nlogn(a:Vec<i32>){
  tails=vec![]; // min tail of length len
  for x in a {
    l=0; r=tails.len();
    while l<r {m=(l+r)/2;
      if tails[m]<x {l=m+1}else{r=m}}
    if l==tails.len(){tails.push(x)}
    else{tails[l]=x}
  }
  tails.len()
}
```

DAG Longest Path

Input `adj: WGraph`. Topological order; max-relaxation; negative weights OK because no cycles.

```
fn dag_longest(adj:WGraph, source:usize){
  n=adj.len(); dist=vec![-INF;n]; parent=vec![None;n]; dist[source]=0;
  for u in topo(adj) {if dist[u]!=-INF {for (v,w) in adj[u] {
    if dist[u]+w>dist[v] {dist[v]=dist[u]+w; parent[v]=Some(u)}
  }}}
  dist
}
```

Runtime $\Theta(V+E)$.

8. Graph Representation / Types

Use these names everywhere:

```
type Graph  = Vec<Vec<usize>>;        // adj[u] = neighbors v
type WGraph = Vec<Vec<(usize,i32)>>;  // adj[u] = (v,weight)
struct Edge { u:usize, v:usize, w:i32 }
type EdgeList = Vec<Edge>;
```

adj list	space $\Theta(V+E)$	scan neighbors $\Theta(\deg u)$
adj matrix	space $\Theta(V^2)$	edge lookup $\Theta(1)$, scan $\Theta(V)$
edge list	space $\Theta(E)$	Bellman-Ford/Kruskal scans all edges

9. BFS

Input `adj: Graph`. Unweighted shortest paths / levels / components / bipartite. Runtime $\Theta(V+E)$.

```
fn bfs(adj:Graph, source:usize){
  n=adj.len(); // shortest path
  dist=vec![INF;n]; //in unweighted graph
  parent=vec![None;n];// levels, components
  queue=VecDeque::new();
  dist[source]=0;
  queue.push\_back(source);

  while let Some(u)=queue.pop_front(){
    for v in adj[u] {
      if dist[v]==INF {
        dist[v]=dist[u]+1;
        parent[v]=Some(u);
        queue.push\_back(v);
      }
    }
  }
  (dist,parent)
}
fn bipartite(adj:Graph){ // 2-coloring
  n=adj.len(); color=vec![-1;n];
  queue=VecDeque::new();
  for s in 0..n {  // false iff odd cycle
    if color[s]!=-1 {continue}
    color[s]=0; queue.push\_back(s);
    while let Some(u)=queue.pop_front(){
      for v in adj[u] {
        if color[v]==-1 {
          color[v]=1-color[u];
          queue.push\_back(v);
        }
        else if color[v]==color[u] {ret false}
      }
    }
  }
  true
}
```

Invariant: queue processed in nondecreasing distance. First discovery gives shortest unweighted distance. If a shorter path to v existed, its predecessor would have been processed earlier, so v would already be discovered.

Uses: unweighted shortest path, levels, connected components, bipartite test.

Path reconstruction: follow parent$[t]$ backward until source, then reverse.

10. DFS / Topological Sort / SCC

Input `adj: Graph`. Colors: WHITE/GRAY/BLACK. Runtime $\Theta(V+E)$.

```
fn dfs(adj:Graph){  // topo, cycle detection, SCC
  n=adj.len(); color=vec![WHITE;n]; parent=vec![None;n]; time=0;
  fn visit(u:usize){
    time+=1; discover[u]=time; color[u]=GRAY;
    for v in adj[u] {
      if color[v]==WHITE {parent[v]=Some(u); visit(v)}
    }
    color[u]=BLACK; time+=1; finish[u]=time;
  }
  for u in 0..n {if color[u]==WHITE {visit(u)}}
}
```

Directed edges: tree = white target; back = gray ancestor; forward/cross = black. Undirected: only tree/back; cycle test = visited neighbor $\neq$ parent. DFS times: d[u]<d[v]<f[v]<f[u] => descendant/tree/forward; d[v]<d[u]<f[u]<f[v] => back to ancestor/cycle; disjoint intervals => cross. Directed cycle detection: edge to GRAY vertex $\Leftrightarrow$ back edge $\Leftrightarrow$ directed cycle. Edge to BLACK does not prove a cycle.

Topological Sort

DAG order: every edge $u \to v$ has u before v. DFS = decreasing finish times.

```
fn topo_dfs(adj:Graph){
  n=adj.len(); seen=vec![false;n]; order=vec![];
  fn visit(u:usize){
    seen[u]=true;
    for v in adj[u]{if !seen[v]{visit(v)}}
    order.push(u)
  }
  for u in 0..n {if !seen[u]{visit(u)}}
  order.reverse(); order
}
fn topo_kahn(adj:Graph){
  n=adj.len(); indeg=vec![0;n];
  queue=VecDeque::new(); order=vec![];
  for u in 0..n {for v in adj[u]{indeg[v]+=1}}
  for u in 0..n {if indeg[u]==0 {
    queue.push\_back(u)}}
  while let Some(u)=queue.pop_front(){
    order.push(u);
    for v in adj[u]{
      indeg[v]-=1;
      if indeg[v]==0{queue.push\_back(v)}
    }
  }
  if order.len()==n {Some(order)} else {None}
}//Kahn returns `None` iff
 //not all vertices are output.
```

Strongly Connected Components

SCC: $u \leftrightsquigarrow v$. Kosaraju = DFS finish order, reverse graph, DFS in decreasing finish order.

```
fn kosaraju(adj:Graph){   // SCC. DFS order ->
  order=dfs_finish_order(adj); rev=reverse_graph(adj);
  seen=vec![false;n]; comps=[]; //-> reverse graph
  for source in order.rev() {
    if seen[source]{continue}
    comp=[]; stack=[source]; seen[source]=true;
    while let Some(u)=stack.pop(){
      comp.push(u);
      for v in rev[u]{
        if !seen[v]{seen[v]=true; stack.push(v)}
      }
    }
    comps.push(comp)
  }
  comps
}
```

Runtime: Theta(V+E).

11. Union-Find / Disjoint Sets

Forest of trees. parent$[x]$ points upward. Root is representative. rank$[x]$ upper-bounds height.

```
struct DSU {
  parent: Vec<usize>,
  rank: Vec<usize>,
}
impl DSU{
  fn new(n){DSU{parent:(0..n).collect(),
    rank:vec![0;n]}}

  fn find(&mut self,x){
    if self.parent[x]!=x {
      self.parent[x]=self.find(self.parent[x])
    }
    self.parent[x]
  }
  fn union(&mut self,x,y){
    let (mut rx,mut ry)=(self.find(x),
      self.find(y));
    if rx==ry {return false}
    if self.rank[rx] < self.rank[ry]
      { swap(rx,ry) }
    self.parent[ry]=rx;
    if self.rank[rx]==self.rank[ry] {
      self.rank[rx]+=1;
    }
    true
  }
}
```

Complexities: LL weighted union O(m + n log n), forest + union by rank + path compression O(m alpha(n)). alpha(n)<= 5 in practice

```
//Connected components:
let mut dsu: DSU = DSU::new(n);
for (u, v) in edges {
    dsu.union(u, v);
}
if dsu.find(a) == dsu.find(b) {
    // same component
}
//Cycle detection (undirected):
let mut dsu: DSU = DSU::new(n);
for (u, v) in edges {
    if !dsu.union(u, v) {
        // cycle
    }
}
```

12. Minimum Spanning Trees

Chooser

sparse graph	Kruskal often easiest
dense graph	Prim often easier
disconnected graph	forest, no MST

tree: connected + acyclic connected graph: tree $\Leftrightarrow$ |E|=|V|-1

MST: spanning tree with min total edge weight

Cut / Cycle Properties

Cut $(S, V - S)$: edge crosses cut iff one endpoint in S, one outside. Light edge = minimum-weight crossing edge.

Cut property: if cut respects chosen edge set A, then a light crossing edge is safe.

Cycle property: for any cycle, a heaviest edge on that cycle is excluded from some MST. If weights distinct, the heaviest edge is in no MST.

Kruskal

```
struct Edge {u,v,w: i32,}
let mut edges: Vec<Edge> = ...;
let mut dsu: DSU = DSU::new(n);
let mut mst: Vec<Edge> = Vec::new();
let mut total: i32 = 0;
fn kruskal(n,edges: Vec<Edge>){ // MST
  sort_by_weight(edges); dsu=DSU::new(n);
  total=0; mst=[];
  for e in edges {    // sort edges + DSU
    if dsu.union(e.u,e.v) {
      total+=e.w; mst.push(e);
      if mst.len()==n-1 {break}
    }
  }
  (total,mst)
}
```

Complexity: sort: O(E log E) DSU: O(E alpha(V)) total: O(E log E)

Output condition: if `mst.len()==n-1`, it is an MST; otherwise graph disconnected and result is a forest.

Traps:

- If graph disconnected $\longrightarrow$ MST does not exist; returns forest.
- Need sort by weight increasing.
- Skip edge if endpoints already connected.

Prim

```
let mut adj: Vec<Vec<(usize, i32)>>
    = vec![Vec::new(); n];
let mut used: Vec<bool> = vec![false; n];
let mut heap: BinaryHeap<Reverse<(i32,
    usize, usize)>> = BinaryHeap::new();
// (weight, vertex, parent)        // MST
fn prim(adj,s){        // min crossing edge
  heap.push((0,s)); total=0; cnt=0;
  while let Some((w,u))=heap.pop_min(){
    if used[u] {continue}
    used[u]=true; total+=w; cnt+=1;
    for (v,w2) in adj[u] {
      if !used[v] {heap.push((w2,v))}
    }
  }
  if cnt!=n {/* disconnected */}
  total
}
```

Complexity:

- Binary heap lazy: $O(E \log E)$; for simple graphs $E \leq V^2$, so $O(E \log V)$.

Output condition: if `cnt==n`, `total` is MST weight; otherwise graph disconnected.

MST Reduction Stencils

Roads+airports: add supernode S; edge S--city cost=airport; old road edges unchanged; MST in augmented graph. Add one new vertex given old MST T: run MST on T plus new incident edges; old non-tree edges still unnecessary by cut/cycle property. Points/routes: complete graph on points, edge cost=squared distance; run Kruskal/Prim. E=Theta(n^2), time O(n^2 log n).

13. Shortest Paths

Chooser

unweighted	BFS
DAG	topo DP
weights ≥ 0	Dijkstra
negative edges	Bellman-Ford

Relaxation

Relax edge (u,v,w): if dist[u]!=INF and dist[v] > dist[u]+w, set dist[v]=dist[u]+w and parent[v]=u.

Dijkstra

```
fn dijkstra(adj:WGraph, source:usize){
  // all weights >= 0
  n=adj.len(); dist=vec![INF;n];
  parent=vec![None;n];
  heap=MinHeap::<(i32,usize)>::new();
  dist[source]=0;
  heap.push((0,source));
  while let Some((du,u))=heap.pop(){
    if du!=dist[u] {continue} // stale
    for (v,w) in adj[u] {
      if dist[v]>du+w {
        dist[v]=du+w; parent[v]=Some(u);
        heap.push((dist[v],v));
      }
    }
  }
  dist
}
```

Runtime $O((V + E) \log V)$. Requires all weights ≥ 0; negative edges break it.

Bellman-Ford

Input `edges: EdgeList`. Negative edges OK; detects reachable negative cycles.

```
fn bellman_ford(n,edges,source){
  dist=vec![INF;n]; parent=vec![None;n];
  dist[source]=0;
  for _ in 0..n-1 { changed=false;
    for e in edges {
      if dist[e.u]!=INF &&
         dist[e.v]>dist[e.u]+e.w {
        dist[e.v]=dist[e.u]+e.w;
        parent[e.v]=Some(e.u);
        changed=true;
      }
    }
    if !changed {break}
  }
  for e in edges {
    if dist[e.u]!=INF &&
      dist[e.v]>dist[e.u]+e.w {
      return None; // reachable neg cycle
    }
  }
  Some(dist)
}
```

Runtime $\Theta(VE)$. Returns `None` iff a reachable negative cycle is detected. Negative cycle not reachable from source: Bellman-Ford still returns shortest paths. Only improvements after pass V reachable from source matter.

Weight transform trap: $w'(e) = a\,w(e)$ with $a > 0$ preserves shortest paths. $w'(e) = w(e) + a$ does not preserve shortest paths when candidate paths use different numbers of edges.

Shortest Paths in DAG

```
fn dag_shortest(adj:WGraph, source:usize){
  let n=adj.len();
  let mut dist=vec![INF;n];
  let mut parent=vec![None;n];

  dist[source]=0;

  for u in topo(adj) {
    if dist[u]==INF {continue}
    for (v,w) in adj[u] {
      if dist[v]>dist[u]+w {
        dist[v]=dist[u]+w;
        parent[v]=Some(u);
      }// topological order.
    }// one relaxation per edge
  }// negative weights ok
  dist
} //Runtime Theta(V+E)
```

Time-Dependent / Ferry Shortest Path

$dist[u]$ = earliest arrival. Road relax: $dist[v] = \min(dist[v], dist[u] + w)$. Ferry/event (u, v, dep, arr) usable iff $dist[u] \leq dep$, then relax $dist[v] = \min(dist[v], arr)$. Use PQ by earliest arrival if time is FIFO/nonnegative; otherwise expand state/event graph.

14. Flow Networks

Directed $G = (V, E)$, source s, sink t, capacities $c(u, v) \geq 0$. Flow constraints: 0<=f(u,v)<=c(u,v); conservation for u!=s,t: inflow=outflow; |f|=out(s)-in(s). Augmenting path: $s \longrightarrow t$ path in residual graph; bottleneck $= \min c_f(e)$ on path. Flow reduction recipe: selected object = unit flow; capacities = usage limits; source/sink = constraints; value = objective; integral caps => integral solution.

Ford-Fulkerson / Edmonds-Karp

Types: `cap[u][v]` residual capacity matrix; `adj: Graph` stores forward+reverse residual neighbors.

```
fn add_edge(u:usize,v:usize,c:i32){
adj[u].push(v);adj[v].push(u);cap[u][v]+=c
}
fn bfs_path(source:usize, sink:usize){
  parent.fill(None); parent[source]=Some(source);
  queue.push\_back((source,INF));
  while let Some((u,flow))=queue.pop_front(){
    for v in adj[u]{
      if parent[v].is_none() && cap[u][v]>0 {
        parent[v]=Some(u);
        pushed=flow.min(cap[u][v]);
        if v==sink {return pushed}
        queue.push\_back((v,pushed))
      }
    }
  }
  0
}
fn max_flow(source:usize, sink:usize){
  flow=0;     // augment while path exists
  loop {
    pushed=bfs_path(source,sink);
    if pushed==0 {break}
    flow+=pushed;
    v=sink;
    while v!=source {
      u=parent[v].unwrap();
      cap[u][v]-=pushed; cap[v][u]+=pushed;
      v=u
    }
  }
  flow
} // f^* is the highest flow
```

Ford-Fulkerson integer caps: $O(E|f^*|)$. Edmonds-Karp BFS: $O(VE^2)$. Final residual graph gives min cut by residual reachability from S.

Max-Flow Min-Cut / Reductions

Max-flow min-cut: $\max |f| = \min c(S, V \setminus S), s \in S, t \notin S$.

Optimality certificate: flow value F and cut capacity F.

Bipartite matching: source$\longrightarrow L$ cap 1; $L \rightarrow R$ cap 1 for original edges; $R \longrightarrow$sink cap 1. Flow-1 $L \rightarrow R$ edges are matched pairs.

Capacitated assignment: $s \longrightarrow$ person cap 1; person$\longrightarrow$ acceptable activity cap 1; activity$\longrightarrow t$ cap capacity. Feasible iff maxflow = #people. Integral capacities give integral assignment.

```
source=0; left0=1; right0=1+n_left; sink=1+n_left+n_right;
for l in 0..n_left {add_edge(source,left0+l,1)}
for (l,r) in bip_edges {add_edge(left0+l,right0+r,1)}
for r in 0..n_right {add_edge(right0+r,sink,1)}
// flow-1 edge left0+l -> right0+r = matched pair
```

Edge-disjoint paths: all edges cap 1. Vertex-disjoint: split v into $v_{in} \longrightarrow v_{out}$ cap 1; original $u \rightarrow v$ becomes $u_{out} \rightarrow v_{in}$ cap INF. Grid domino: checkerboard cells L/R, edge for adjacency, max dominoes = max bipartite matching. Path $x \rightarrow y$ through z: need two internally vertex-disjoint paths $z \rightsquigarrow x$ and $z \rightsquigarrow y$; split vertices cap 1, add $x, y \rightarrow t$ cap 1, ask maxflow $z \rightarrow t$ value 2. Special edge $e = ab$ used: try orientations $\cdots a - b \cdots$ and $\cdots b - a \cdots$; remove e, enforce disjoint prefix/suffix with unit capacities.

15. Randomized / Probabilistic Analysis

Indicator: $X_A = 1[A]$, $\mathbb{E}[X_A] = \Pr(A)$. Linearity: $E[\Sigma_i X_i] = \Sigma_i E[X_i]$. No independence needed.

Hiring: $P(i \text{ new best}) = 1/i$, so $E[\#hires] = \Sigma_i 1/i = H_n = \Theta(\log n)$. Hat-check: $P(i \text{ own hat}) = 1/n$, so $E[\#fixed] = 1$.

Randomized quicksort: random pivot avoids input-dependent bad order; expected $\Theta(n \log n)$, worst still $\Theta(n^2)$.

Birthday: $\Pr[\text{all different among } q \text{ into } m] = \Pi_{i=0}^{q-1}(1 - i/m) \approx e^{-q(q-1)/(2m)}$; for 365 days, $q = 23$ gives match prob $> 1/2$. Integral bound for decreasing positive f: $\Sigma_{k=a+1}^{b} f(k) \leq \int_a^b f(x)\,dx$.

16. Online Algorithms / Competitive Analysis / Experts

Online: decide before future known. Competitive ratio: $ALG(I) \leq r\,OPT(I) + c$. Ski rental: rent 1/day, buy B. Rent until paid B, then buy. If $T \leq B$: ALG=T=OPT. If $T > B$: ALG<=B+B=2B=2OPT. Lower bound: adversary stops just before/after purchase; deterministic CR=2.

Caching MC traps

LRU evicts least-recently requested page. LFU evicts smallest request count; tie usually LRU. Neither has pointwise fewer misses than the other on every sequence. LRU is k-competitive for cache size k; no deterministic paging algorithm beats k in worst-case competitive ratio.

Experts / Weighted Majority

n experts, rounds $t = 1..T$. Expert i predicts; loss $\ell_{i,t} \in \{0, 1\}$. Weights $w_i = 1$ initially. Let $W = \Sigma_i w_i$.

```
for t in 1..=T {
  predict weighted majority of experts;
  observe true outcome;
  for expert i wrong this round {
    w[i] *= beta;        // 0 < beta < 1, often beta=1/2
  }
}
```

Weighted prediction: choose label with larger total weight. Potential: $W_t = \Sigma_i w_i^{(t)}$. If ALG makes mistake, wrong-weight $\geq W_t/2$, so $W_{t+1} \leq W_t/2 \cdot \beta + W_t/2 = W_t(1+\beta)/2$. If best expert has m^* mistakes, its weight is β^{m^*}, hence $\beta^{m^*} \leq W_T \leq n((1+\beta)/2)^M$. For $\beta = 1/2$: $M \leq 2.41(m^* + \log_2 n)$.

General randomized Hedge: choose expert with prob w_i/W; update $w_i \leftarrow w_i e^{-\eta \ell_{i,t}}$; regret $O(\sqrt{T \log n})$.

Prophet Inequality

Independent nonnegative X_i, known distributions. $M = \max_i X_i$, $z = E[M]$, threshold $\tau = z/2$. Accept first $X_i \geq \tau$. Guarantee $E[ALG] \geq z/2$. Proof stencil: $q = P[\forall i : X_i < \tau]$, $b_i = E[(X_i - \tau)^+]$. Value from threshold part $\geq \tau(1 - q)$; overshoot part $\geq q\,\Sigma_i b_i$; combine to get $\geq z/2$.

Secretary: random order; observe first n/e candidates, remember best rank seen, then hire first later candidate better than all observed.

$$P(\text{success}) \approx 1/e$$

17. WORKED EXAM PROBLEMS

P01. Loop invariant proof: linear search returns matching index or NIL.
Inv.: before iteration i, no earlier position contains v: A[j] != v for all j<i. Init: i=1: empty set. Maint: if A[i] != v, then the statement extends to all j<i+1. Term: if equality found, returned index correct; if loop finishes, invariant says no position contains v.

P02. Asymptotic proof with constants: show (n+a)^b = Theta(n^b) for fixed constants.
For n>=2|a|: n/2 <= n+a <= 2n. Raise to fixed b>0: 2^(-b) n^b <= (n+a)^b <= 2^b n^b. Choose constants c1=2^(-b), c2=2^b, n0=2|a|.

P03. Recursion tree: T(n)=8T(n/2)+Theta(n^2), leaves dominate.
Level i: 8^i nodes, size n/2^i, work 8^i*c*(n/2^i)^2 = c*n^2*2^i. Sum until lg n: c*n^2*sum_i=0..lg n 2^i = Theta(n^3). Leaves: 8^(lg n)=n^3. Hence Theta(n^3).

P04. Recursive insertion sort: remove last element, sort prefix, insert last.
Recurrence T(n)=T(n-1)+Theta(n). Unroll: T(n)=Theta(sum_i=1..n i)=Theta(n^2). Correctness by induction: sorted prefix by IH; insertion puts last element in unique sorted position.

P05. Strengthened recurrence proof: split sizes 3n/5, 4n/5 plus linear work.
T(n)=T(3n/5)+T(4n/5)+a*n. Guess Theta(n^2) since (3/5)^2+(4/5)^2=1. Upper needs slack: IH T(m)<=c*m^2-d*m. Then T(n)<=c*n^2-d*(7n/5)+a*n <= c*n^2-d*n iff d>=5a/2. Pick d, then pick c for bases. Lower: T(n)>=c*((3n/5)^2+(4n/5)^2)=c*n^2.

P06. Master/Akra pattern: choose fastest among several recurrence-defined algorithms.
Compute exponent. aT(n/b)+n^k: compare k to log base b of a. Examples: 2T(n/2)+n^3 -> Theta(n^3); 30T(n/4)+n -> Theta(n^(log_4 30)); 8T(n/2)+1 -> Theta(n^3); T(n/2)+n^4 -> Theta(n^4); 2T(n-1)+1 -> Theta(2^n). Pick smallest asymptotic.

P07. Restaurant spacing DP: choose subset of line positions at mutual distance at least k.
Sort by position. dp[i] = max profit using restaurants i..n. next(i) = first j>i with M[j]>=M[i]+k. dp[n+1]=0, dp[i]=max(dp[i+1], p[i]+dp[next(i)]). Ans: dp[1]. Proof: optimum skips i or takes i, then all remaining choices start at next(i). Runtime O(n log n) with binary search.

P08. Restaurant variant: force the current restaurant to be the leftmost open one.
c[i] = best profit among solutions whose leftmost chosen restaurant is i. Then c[i]=p[i]+max{c[j]: j>i and M[j]>=M[i]+k}, with empty max 0. Ans: max_i c[i]. Use suffix maxima plus binary search for O(n log n), or scan for O(n^2).

P09. Binary search proof: sorted array membership in O(log n).
Maintain half-open interval [l,r) with invariant: if x is present then it lies in A[l..r). Compare with A[m]. If A[m]<x, discard [l,m]; if A[m]>x, discard [m,r). Sortedness preserves invariant. Termination gives found index or empty candidate set. Runtime T(n)=T(n/2)+O(1).

P10. 2D sorted matrix: rows and columns increasing, find x in O(n).
Start at top-right (i=1,j=n). If A[i,j]=x return. If A[i,j]>x, move left: everything below in column j is also >x. If A[i,j]<x, move down: everything left in row i is also <x. Each step removes one row/column, so at most 2n-1 steps.

P11. Augmented BST interval counting: count keys in [a,b] with subtree sizes.
Store size(x)=1+size(L)+size(R) and update on insert/delete rotations/path. rankLT(x) follows search path; when moving right add size(left)+1. rankLE(x) similarly includes equals. Count [a,b]=rankLE(b)-rankLT(a). Runtime O(h).

P12. MinStack: stack with Push, Pop, GetMin all worst-case O(1).
Use two stacks: data stack S, minimum stack M. Push x: push to S; if M empty or x<=top(M) push x to M. Pop: y=pop(S); if y=top(M) pop M. GetMin returns top(M). Duplicates need <= on push, not just <.

P13. Heap / priority queue trace: Build-Max-Heap and Heapify correctness.
BuildHeap calls Heapify from floor(n/2) down to 1. Invariant: before processing i, all nodes >i root heaps. Heapify assumes children heaps and pushes violation downward. Runtime sum over heights h of (n/2^(h+1))*O(h)=O(n), not O(n log n).

P14. Comparison sorting lower bound: prove Omega(n log n) comparisons.
Any deterministic comparison sort is a decision tree. Each permutation of n distinct keys must end in a different leaf, so at least n! leaves. Height h satisfies 2^h >= n!, so h>=log2(n!)=Omega(n log n).

P15. Randomized Quicksort comparison probability relative to a fixed element.
For ranks i<j, they are compared iff the first pivot chosen among ranks i..j is either i or j. Thus P(i,j compared)=2/(j-i+1). For element of rank r, probability with rank q is 2/(|q-r|+1); sort options by distance from r.

P16. All-equal Lomuto partition: bad split and robust duplicate fix.
If partition condition is <= pivot, all equal elements move left, pivot index returns r. Recurrence T(n)=T(n-1)+Theta(n)=Theta(n^2). Fix with 3-way partition into <,=,> pivot or choose middle of equal block.

P17. Simple uniform hashing: expected number of pair collisions.
X=sum over pairs i<j of X[i,j], where X[i,j]=1[h(i)=h(j)]. E[X[i,j]]=1/m. Thus E[X]=C(n,2)/m=Theta(n^2/m). For collisions with first slot only: pair probability 1/m^2, so expectation C(n,2)/m^2=Theta(n^2/m^2).

P18. Hash values are length-ell arrays over m symbols: expected collisions at most 1.
For fixed distinct x,y, collision requires equality in all ell coordinates, so P[x,y collide]=(1/m)^ell. Let X=sum over pairs x<y of X[x,y]. Then E[X]=C(n,2)/m^ell <= n^2/(2m^ell) <= 1 if m^ell >= n^2.

P19. Hash chains high-probability bound: no bucket longer than ell.
A k-collision is a k-set in one bucket. E[# k-collisions] <= C(n,k)/m^(k-1) <= n^k/m^(k-1). Chain length >ell iff an (ell+1)-collision exists. If m>=100n^(1+1/ell), then E[#(ell+1)-collisions] <= n^(ell+1)/m^ell <= 1/100^ell <=0.01. Markov: bad prob <=0.01.

P20. Expected hires with random arrival, skip first 10 percent.
For position i>n/10, indicator X[i]=1 if candidate i is best among first i. P[X[i]=1]=1/i. Expected hires = sum_i=n/10+1..n 1/i <= integral from n/10 to n of dx/x = ln 10 < 10.

P21. Random edge/color expectation: count bad local events by indicators.
If each edge/vertex color is independent uniform among q colors, define X[e]=1 if event e is bad. Usually P[X[e]=1] is 1/q or 1/q^r. Then E[#bad]=sum_e P[X[e]=1]. Use Markov P[#bad>=1] <= E[#bad] for existence/probability bounds.

P22. Bipartite matching as unit-capacity max-flow.
Build s->L cap 1; original edges L->R cap 1; R->t cap 1. Integral capacities give integral max flow. Capacity 1 forces each vertex used once. Flow edge l->r is a matched pair; every matching gives same-size flow. Hence max flow = max matching.

P23. Capacitated activity/cheese assignment: each person gets one acceptable item, bins have capacities.
Network: s->each person/type cap demand; person/type->item if acceptable cap INF or demand; item->t cap supply/spots. Feasible iff maxflow equals total people/students. Integral flow gives integer assignment; assignment gives flow.

P24. Domino tiling / maximum non-overlapping adjacent grid pairs.
Checkerboard color cells. Black cells L, white cells R. For every adjacent allowed pair add edge L->R. Max domino placement = maximum bipartite matching: each domino is one matched edge and matching prevents cell reuse.

P25. Edge-disjoint paths: route multiple paths without reusing edges.
Give every edge capacity 1 and run maxflow. Value k iff there are k edge-disjoint paths (integrality). For undirected edges, either replace by two directed arcs sharing one unit-capacity gadget, or orient both ways carefully with a single capacity-1 edge gadget.

P26. Vertex-disjoint paths / simple path through required vertex z.
To find simple path x->z->y, need internally vertex-disjoint paths from z to x and z to y. Split every vertex v into v.in->v.out cap 1 (INF for terminals if allowed). Original edge u-v becomes arcs u.out->v.in and v.out->u.in. Add x,y->t cap 1, source z, ask maxflow value 2.

P27. Special required edge e*={a,b} in an edge-simple u-v trail.
Try both orientations. For orientation a->b, remove e* and require edge-disjoint prefix/suffix paths u->a and b->v using unit edge capacities; similarly orientation b->a. If flow construction returns both paths, concatenate prefix + e* + suffix. If both orientations fail, no valid trail.

P28. Haunted-house escape: k agents, grid over times, ghosts, no two agents same cell.
Build time-expanded flow. For every safe cell (c,t) create cell(c,t).in->cell(c,t).out cap 1. For each legal stay/move c->c' add cell(c,t).out->cell(c',t+1).in cap 1 if target safe. Source to start cells at time 1 cap 1. Boundary cells at times <=T connect to sink cap 1. Maxflow=k iff all escape. Size O(Tn^2) nodes/edges up to constants.

P29. Shortest path with battery/resource budget: minimize time, spend at most B.
State graph has vertices (v,b) = at v after spending b battery, 0<=b<=B. For edge e=(u,v) add (u,b)->(v,b+batt(e)) with weight time(e) if b+batt(e)<=B (both directions if undirected). Run Dijkstra from (s,0). Ans: min over b<=B of dist(t,b). Runtime O(BE log(BV)).

P30. Road network mutual reachability forbidden for listed directed pairs.
Compute SCCs with Kosaraju/Tarjan in O(V+E). For each forbidden pair (u,v), reject iff comp[u]=comp[v]. Same SCC iff u->v and v->u are both reachable. If all pairs pass, accept. Given ell<=E, total runtime O(V+E).

P31. Topological-sort / Course Schedule: can all prerequisites be satisfied?
Directed graph prerequisite u->v. Run DFS colors or Kahn indegrees. If cycle/back edge exists, impossible. Otherwise topo order exists and gives valid schedule. Runtime O(V+E). Proof: DAG iff topo order iff no DFS back edge.

P32. DAG maximum-weight path with vertex weights from s to t.
Topologically sort. Initialize dp[s]=w(s), others -INF. In topo order, relax u->v: dp[v]=max(dp[v], dp[u]+w(v)). Ans: dp[t]. Correct because all predecessors of v processed before v. Runtime O(V+E).

P33. Verify claimed shortest-path distances in graph with negative edges but no negative cycles.
Check d[s]=0. For every edge (u,v) check triangle inequality d[v]<=d[u]+w(u,v); if violated reject. Build tight-edge graph E'={(u,v): d[v]=d[u]+w(u,v)}. DFS/BFS from s in E'. Accept iff all vertices reached. Inequality gives d[v] <= every path length; tight paths give actual paths of length d[v], so equality.

P34. Bellman-Ford negative-cycle / altitude consistency with signed edge changes.
A bad closed tour is a directed cycle with nonzero total weight. Run Bellman-Ford on w and on -w from a super-source connected to all vertices by 0 edges. A negative cycle in w or in -w means some cycle has nonzero sum. If neither run detects, all cycles have sum 0. Runtime O(VE).

P35. Shortest-path weight transform MCQ: add constant vs multiply by positive constant.
Multiplying all edge weights by a>0 preserves every path ordering: w(P) becomes a*w(P). Adding a to every edge changes w(P) to w(P)+a*|P|, so paths with different numbers of edges can change order.

P36. Dijkstra choice proof: nonnegative weights only.
Maintain finalized set S. Pick outside vertex x with minimum tentative distance. Any s->x path first crosses from S to V-S at (u,v); length to v is at least d[u]+w(u,v)>=d[v]>=d[x] because w>=0. Thus d[x] is final. Negative edges break this.

P37. Bellman-Ford exact invariant and negative-cycle certificate.
After i full passes, d[v] is shortest path length among paths using at most i edges. Induct by last edge. Without negative cycles, shortest paths can be simple, so use at most V-1 edges. If a V-th pass still relaxes, there is a reachable negative cycle.

P38. MST over complete graph of points: connect all islands with n-1 routes.
Build complete undirected graph on the n points with edge weight squared Euclidean distance. Need connected graph with n-1 routes minimizing total cost = MST. Run Kruskal/Prim on O(n^2) edges. Runtime O(n^2 log n); output MST edges.

P39. MST cut/cycle proof: min crossing edge safe, max cycle edge unsafe.
Cut: add lightest crossing edge e to an MST not using it; cycle contains another crossing edge f with w(f)>=w(e); swap f->e. Cycle: if unique heaviest edge e of a cycle is in MST, remove it; another cycle edge reconnects the cut cheaper, contradiction.

P40. Minimum-weight feedback edge set in undirected weighted graph.
Need remove edges so remaining graph is acyclic. Equivalent: keep an acyclic subset of maximum total weight. In connected graph, max kept acyclic set is a maximum spanning tree. Run Kruskal in descending weights, keep tree edges, remove all others. Correct because any acyclic connected kept set has at most V-1 edges and maximum spanning tree maximizes kept weight.

P41. Airport + roads connectivity: cities can connect by roads or both having airports.
Add super-node r. For every city v, add edge (r,v) of cost airport cost c[v]. Keep original road edges with costs w[e]. Run MST on augmented graph. Chosen original edges = roads; chosen r-edges = airports. Super-node models free travel between airports. Runtime O((E+V)log(E+V)).

P42. Edge belongs to at least one MST: linear-time query for edge e=(a,b).
Let W=w(e). Run DFS/BFS from a using only edges with weight <W. If b reached, then e is heaviest on a strictly lighter cycle, so no MST contains it. If not reached, take component S of a in subgraph w<W; every edge crossing (S,V-S) has weight >=W, and e crosses, so e is lightest for this cut. Thus some MST contains e. Runtime O(V+E).

P43. Unique MST vs many MSTs: detect swappable equal-weight non-tree edge.
Compute one MST T. For every non-tree edge e=(u,v), find maximum edge weight M on the unique path u->v in T. If w(e)=M, swap e with a max edge on that path to get another MST: MANY. If all w(e)>M, no equal swap exists: ONE. Naive runtime O(EV); faster with LCA if needed.

P44. Connected graph with n vertices and n edges: count spanning trees in O(n).
Connected with |E|=|V| has exactly one cycle. Find cycle length c by DFS back edge or repeatedly delete degree-1 leaves until the cycle remains. Every spanning tree is obtained by deleting exactly one cycle edge; deleting non-cycle edge disconnects. Ans: number of spanning trees = c, hence between 3 and n for simple graph.

P45. Tree diameter: output farthest pair of nodes in binary tree.
Postorder returns (height, deepest, bestLen, bestPair). At node u, candidate crossing pair is deepest node in left subtree to deepest node in right subtree with length hL+hR+2 (handle empty child). Best is max of left best, right best, crossing. Runtime O(n).

P46. Longest simple path constrained to pass through the root.
Compute heights of left and right subtrees. If height counts edges and empty height is -1, answer is h(root.left)+h(root.right)+2. The path through root is maximized by independently choosing deepest descendant on each side. Runtime O(n).

P47. Reconstruct binary tree from inorder and postorder traversals.
Root is last element of postorder. Find root in inorder; left/right inorder blocks determine subtree sizes. Split postorder into corresponding left/right blocks, recurse. Base empty list. Runtime O(n) with value-to-inorder-index map. Preorder+inorder similar; preorder+postorder alone is not unique.

P48. Travel passes DP: cover sorted travel days with 1/7/30-day passes.
Days d[1]<...<d[n], lengths L, costs c[L]. dp[i] = min cost to cover days i..n. next(i,L)=first j with d[j]>=d[i]+L. dp[n+1]=0, dp[i]=min over L of c[L]+dp[next(i,L)]. Fill decreasing i. Ans: dp[1]. Trap: pass bought on day d[i] covers days <d[i]+L.

P49. Weighted interval scheduling: choose non-overlapping intervals of max weight.
Sort by finish. p(i)=last interval ending <=start[i]. dp[i]=best using first i intervals. dp[0]=0, dp[i]=max(dp[i-1], w[i]+dp[p(i)]). Ans: dp[n]. Runtime O(n log n) with binary search for p.

P50. Cardinality-constrained weighted intervals: choose at most k disjoint intervals.
Sort by finish and compute p(i). dp[i][j]=best value using first i intervals and at most j intervals. Base dp[0][j]=dp[i][0]=0. Recurrence dp[i][j]=max(dp[i-1][j], w[i]+dp[p(i)][j-1]). Ans: dp[n][k]. Runtime O(nk+n log n).

P51. Interval scheduling greedy: max number of mutually disjoint intervals.
Sort by increasing finish time. Repeatedly take first interval compatible with current last finish. Exchange proof: earliest-finishing interval e can replace first interval of any optimum without reducing feasibility. Then recurse on intervals starting after e.

P52. Divide farm into exactly 3 contiguous nonempty parts minimizing largest sum.
Input A[1..n], A[i] positive and A[i]<=1000. Need cuts i<j minimizing max(sum 1..i, sum i+1..j, sum j+1..n). Binary-search answer M = allowed max child area. Bounds: L=max A[i] (cannot put A[i] anywhere if M<A[i]), R=sum A[i] (one part contains all). Feasible(M): scan left->right with cur=0, parts=1; if cur+x<=M then cur+=x, else parts++, cur=x. Reject if parts>3. Why greedy works: for fixed M, greedy makes each part as long as possible, so after t parts it has covered at least as much prefix as any valid t-part partition; hence greedy uses the minimum possible #parts. Therefore parts>3 => impossible. If parts<=3 => possible for exactly 3: n>=3 and all A[i]>0, so split existing nonempty parts until there are 3; splitting only decreases part sums. Predicate monotone: bigger M cannot need more parts. Binary search smallest feasible M. Runtime Feasible=O(n), R<=1000n => O(n log(1000n))=O(n log n). No direct sum(a,b) needed; cur is computed during scan. Recover cuts: rerun greedy at M*, then split if fewer than 3 parts, output the two cut indices.

P53. Split array into k contiguous parts minimizing maximum load.
Same as P52 with k instead of 3. L=max a[i], R=sum a[i]. Feasible(M) greedily computes the minimum number q of parts needed with each part sum<=M. Feasible iff q<=k; if exact k required and n>=k, split parts until k. Binary search smallest feasible M. Runtime O(n log R); if a[i] polynomially bounded, O(n log n).

P54. Short grid walk with value and move limit: moves right/up/diag, at most 3n/2 moves.
dp[i][j][ell] = max value reaching cell (i,j) in exactly ell moves. Base dp[0][0][0]=v(0,0). Transition from predecessors (i-1,j), (i,j-1), (i-1,j-1) with ell-1, add v(i,j). Ans: max over ell<=floor(3n/2) of dp[n][n][ell]. Runtime O(n^3).

P55. Rod cutting / unbounded segmentation: maximize revenue over cuts summing to n.
r[0]=0. For length j=1..n: r[j]=max over 1<=i<=j of p[i]+r[j-i]. This chooses leftmost piece length i and solves remainder optimally. Runtime O(n^2), space O(n). Store argmax to reconstruct cuts.

P56. Matrix-chain multiplication: choose parenthesization minimizing scalar multiplications.
Matrices A[i] have dimensions p[i-1] x p[i]. m[i,i]=0. For length ell=2..n: m[i,j]=min over i<=k<j of m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]. Ans: m[1,n]. Runtime O(n^3), space O(n^2).

P57. LCS of two strings: subsequence, not substring.
dp[i][j] = LCS length of prefixes X[1..i], Y[1..j]. Base row/col 0. If X[i]=Y[j], dp[i][j]=1+dp[i-1][j-1]; else max(dp[i-1][j], dp[i][j-1]). Runtime O(nm). Reconstruct by walking backward.

P58. Longest increasing subsequence: standard DP and faster tails version.
DP: L[j]=1+max{L[i]: i<j and A[i]<A[j]}, empty max 0; answer max_j L[j], O(n^2). Tails: maintain smallest possible tail for each length; for x, lower_bound first tail >= x and replace. Length of tails = LIS length, O(n log n).

P59. Longest modular increasing subsequence: increasing values and sum divisible by 3.
r[j,b] = best length of increasing subsequence ending at j with sum mod 3 equal b. Initialize r[j,A[j] mod 3]=1. For all i<j with A[i]<A[j] and all b: r[j,(b+A[j]) mod 3]=max(r[j,(b+A[j]) mod 3], r[i,b]+1). Ans: max_j r[j,0]. Runtime O(n^2).

P60. 0/1 knapsack: each item used at most once, capacity W.
dp[i][w] = best value using first i items with capacity w. dp[0][w]=0. If weight[i]>w, skip; else dp[i][w]=max(dp[i-1][w], value[i]+dp[i-1][w-weight[i]]). Runtime O(nW) pseudo-polynomial. 1D update w decreasing.

P61. 3-way equal partition: split positive integers into 3 subsets each sum W/3.
If W mod 3 != 0, NO. Let target S=W/3. dp[i][x][y] = possible after first i items with subset1 sum x, subset2 sum y; subset3 implicit. Transition: put item i in subset1 if x>=w[i], subset2 if y>=w[i], or subset3. Ans: dp[n][S][S]. Runtime O(nS^2).

P62. Coin-row two-player game: optimal play, take leftmost or rightmost coin.
dp[i][j] = max value current player can guarantee from interval i..j. dp[i][i]=a[i]. Recurrence dp[i][j]=max(a[i]+min(dp[i+2][j],dp[i+1][j-1]), a[j]+min(dp[i+1][j-1],dp[i][j-2])). Opponent chooses branch minimizing our future. Fill by interval length.

P63. Coin change / minimum number of coins, unlimited copies.
dp[0]=0, dp[x]=INF initially. For amount x=1..W: dp[x]=1+min over c<=x of dp[x-c]. Ans: dp[W] or impossible if INF. Runtime O(W*#coins). For number of ways, loop coins outside and amounts increasing.

P64. Optimal BST DP with key probabilities / search frequencies.
e[i,j] = min expected cost for keys i..j, w[i,j] = total probability/frequency in interval. Choose root r: e[i,j]=min over r=i..j of e[i,r-1]+e[r+1,j]+w[i,j]. Empty interval cost 0. Fill by increasing length. Runtime O(n^3).

P65. DP answer skeleton when the problem statement is unfamiliar.
State meaning with units and exact interval/index convention. Base cases. Transition: enumerate the first/last decision and combine with smaller solved states. Order: dependencies first. Answer: exact table cell. Correctness: induction in fill order; optimum uses one enumerated choice and every candidate is feasible. Runtime = states * transition cost.

P66. Ski rental competitive analysis: rent price 1, buy price B.
Algorithm: rent until total rent reaches B, then buy. If T<=B, ALG=T=OPT. If T>B, ALG<=B+B=2B=2OPT. Lower bound for deterministic algorithms: adversary stops just before algorithm buys or continues just after; no deterministic ratio <2.

P67. Paging/caching LRU: cache size k, prove k-competitive.
Partition requests into phases with at most k distinct pages; new phase starts when (k+1)-st distinct page appears. LRU has at most k misses per phase. OPT has at least 1 miss per completed phase because phase plus next first request contains k+1 distinct pages. Hence LRU <= k*OPT + O(k).

P68. Weighted majority / experts: bound mistakes against best expert.
Weights start 1; after wrong prediction multiply expert weight by beta. If algorithm makes a mistake, wrong side has weight >=W/2, so W'<=W(1+beta)/2. Best expert with m* mistakes has weight beta^(m*) <= W[T] <= n((1+beta)/2)^M. Solve for M.

P69. Secretary problem: random order, choose maximum candidate online.
Reject first r=floor(n/e), remember best sample. Then accept first candidate better than all seen. Success if global best appears at position i>r and best among first i-1 was in sample. Probability sum_i=r+1..n (1/n)*(r/(i-1)) approx (r/n)ln(n/r) approx 1/e.

P70. Online bipartite matching: greedy and ranking guarantees.
Greedy matches each arriving right vertex to any free neighbor. Each ALG edge can be charged by at most two OPT edges, so ALG >= OPT/2. Ranking: pick random priority order on offline left vertices; each arrival chooses highest-priority free neighbor; expected value (1-1/e)OPT.

P71. Prophet inequality: threshold at half expected maximum.
Independent nonnegative X[i], known distributions. Let M=max_i X[i], z=E[M], threshold tau=z/2. Accept first X[i]>=tau. Guarantee E[ALG]>=z/2. Proof uses q=P[all i: X[i]<tau] and overshoots (X[i]-tau)^+; threshold contribution tau(1-q) plus overshoot contribution q*sum_i E[(X[i]-tau)^+].

Standard / LeetCode Micro-Stencils

Two Sum: scan i left->right; map has earlier values j<i. If target-a[i] exists return; else insert. Inv: when second endpoint appears, first is stored. Exp O(n).
Valid parentheses: stack unmatched opens; closing bracket must match top; accept iff stack empty. Inv: stack = opens not yet closed, in order. O(n).
Grid islands: for each unvisited land cell: count++, BFS/DFS 4-neighbors. Each cell/edge checked O(1) times. O(nm).
Redundant connection: DSU scan edges in input order; if find(u)==find(v), current edge closes first cycle; else union. O(E alpha V).
Top-k frequent: count HashMap. Heap: O(n+m log m), pop k. Bucket by freq 0..n: append to bucket[f], scan downward => O(n+m).
Distinct sliding window: l=0; last map. For r: if last[a[r]]>=l set l=last+1; update ans=r-l+1. Inv: window [l,r] duplicate-free and longest ending at r. Exp O(n).
K sorted lists: min-heap first element of each list; pop smallest, push next from same list. N elems, heap size k => O(N log k), space O(k).
Max subarray: D&C: left/right/cross(best suffix+prefix), T=2T(n/2)+O(n). Kadane: cur=max(a[i],cur+a[i]); best=max(best,cur). O(n).
Count inversions: merge sort; when R[j] copied before L[i..], add remaining left count. Correct because those are exactly cross inversions. O(n log n).
Merge stability proof: merge inv: output prefix sorted and contains smallest consumed elems. Tie -> take left first, preserving original order.
Quickselect: random pivot; recurse only into side containing kth. Good pivot shrinks to <=3n/4 with const prob => expected O(n).
BFS proof: queue nondecreasing dist. If shorter path to v existed, predecessor on shorter path processed earlier and would have discovered v.
Multi-source BFS: push all sources with dist=0. Same as super-source to all sources. First pop gives nearest-source distance. O(V+E).
DFS/topo/cycle: directed edge to GRAY ancestor = cycle. If no back edge, decreasing finish times topo-sort since every edge u->v has f[u]>f[v].
Undirected cycle: DFS: visited neighbor != parent => cycle. DSU: edge inside same component closes cycle.
SCC condensation: components as nodes; cross-component edges form DAG. A cycle of components would merge into one SCC. Use topo DP on condensation.
Kruskal ties: equal-weight tie order may change tree, not weight. Accepted edge still lightest crossing a component cut, so safe.
Second-best MST: for each non-tree e=(u,v), add e to T, remove max-weight edge on T-path u..v. Candidate w(T)+w(e)-maxpath. Min candidate >w(T). Naive O(EV).
Min cut from maxflow: final residual BFS from s gives S. Cut edges u in S, v notin S. Crossing S->T saturated and T->S zero, so cap=cut=value.
Project selection: profit p_v, dependency u=>v. s->v cap p_v if p_v>0; v->t cap -p_v if p_v<0; u->v cap INF. Max profit = positiveSum - mincut.
Interval partitioning: sort by start; min-heap room finish times. If minFinish<=s reuse else new room. Max heap size = max overlap = optimum. O(n log n).
Sweep max overlap: events (start,+1),(end,-1); for half-open [s,e), process end before start at tie. Running max active = overlap.
LRU data struct: HashMap key->DLL node; list by recency. Get/move front. Put/update front; evict tail. Exp O(1).
LFU data struct: key->(val,freq,node), freq->DLL, minFreq. On access move key f->f+1; evict LRU node in minFreq list. Exp O(1).
LRU vs LFU: neither dominates pointwise. LFU can keep old frequent page; LRU can evict frequent page after long scan. Course fact: LRU k-competitive.
Anagrams: key = sorted chars or frequency vector. Same multiset iff same key. HashMap key->group. O(sum L log L) or O(sum L) fixed alphabet.
Valid anagram: same length and all char counts equal; count first, subtract second, reject negative/nonzero. O(n).
House robber path: dp0=0, dp1=a1, dp[i]=max(dp[i-1], a[i]+dp[i-2]). Choice: skip/take i. O(n), O(1) space.
Tree independent set: postorder returns (take,skip). take(u)=w_u+sum skip(ch); skip(u)=sum max(take,skip). Ans max at root.
Word break: dp[0]=true; dp[i]=OR_j<i(dp[j] and s[j..i] in Dict). Ans dp[n]. O(n2) checks plus substring cost.
Edit distance: dp[i][j]=min edits A[1..i]->B[1..j]. Base dp[i][0]=i, dp[0][j]=j. Equal -> diag; else 1+min(del,ins,rep). O(nm).
SP with <=K edges: layered BF: dp[l][v]=best with <=l edges. Init dp[0][s]=0. For l=1..K relax all edges from dp[l-1]. O(KE).
SP with one discount/free edge: state (v,used). Normal edge keeps used; discount transition only from used=0 to used=1. Run Dijkstra if costs nonnegative.
BST select kth: L=size(left). If k=L+1 return root; if k<=L go left; else go right with k-L-1. O(h).
Validate BST: valid(x,lo,hi): require lo < x.key < hi, recurse left (lo,x), right (x,hi); adapt duplicate convention. O(n).
DAG count paths: topo; ways[s]=1; for u in order, for edge u->v: ways[v]+=ways[u]. All preds processed before v. O(V+E).
Prefix sums: pref[0]=0; pref[i+1]=pref[i]+a[i]. Range sum [l,r)=pref[r]-pref[l]. Use for fast interval cost in DP.
Binary-search answer: choose feasible(M), prove monotone and exact. Maintain smallest feasible in [L,R]. If feasible(mid) R=mid else L=mid+1.

More Standard Problem Micro-Stencils

Merge intervals: Sort by start. Keep last output [l,r]. If next.s<=r (closed/touching convention) merge r=max(r,next.e), else output new interval. O(n log n).
Remove overlaps: Max keep = interval scheduling by earliest finish. Min removals = n-kept. Proof: earliest finish leaves largest remaining suffix.
Interval stabbing / arrows: Sort by end. Place point/arrow at end of first uncovered interval; delete all intervals containing it. Exchange: move OPT's first point to that end.
Meeting rooms possible: Sort by start; conflict iff some next.start < current.end (for half-open [s,e), equality OK). Equivalent max overlap<=1.
Two-pointer sorted sum: Sorted A. l=0,r=n-1. If A[l]+A[r]<T increase l; if >T decrease r; else found. Monotonicity proves discarded pair impossible.
3Sum: Sort. Fix i; two-pointer l=i+1,r=n-1 for target -A[i]. Skip duplicate i/l/r values. Runtime O(n^2).
Subarray sum K: Prefix p. Count previous prefixes equal p-K in HashMap. Add count before inserting current p. Works with negative numbers. O(n) expected.
Positive window target: If all numbers positive/nonnegative, expanding r increases sum and moving l decreases it; two pointers can find/count target or <=K windows. Not valid with negatives.
Sliding-window max: Deque of indices with decreasing values. Drop out-of-window front; while back value <= new value pop back; front is max. Each index enters/exits once => O(n).
Median maintenance: Max-heap lower half, min-heap upper half. Rebalance sizes differ by <=1 and all lower<=upper. Median is root(s). Insert O(log n).
Kth largest stream: Min-heap size k. For each x push; if size>k pop min. Heap root is kth largest of seen items. O(n log k), space O(k).
2D prefix sums: P[i+1][j+1]=A[i][j]+P[i][j+1]+P[i+1][j]-P[i][j]. Rect [r1,r2)x[c1,c2): four-corner inclusion-exclusion.
Difference array: For range add x on [l,r): diff[l]+=x, diff[r]-=x. Prefix diff to materialize final array. 2D uses four corner updates.
Trie / prefix tree: Edges are characters. insert/search/startsWith are O(word length). Useful for prefix dictionaries; terminal flag distinguishes word vs prefix.
Kahn topo-sort: Compute indegrees; queue all indeg-0 vertices; pop u and decrement outgoing neighbors. If output size<V, cycle exists. O(V+E).
Alien dictionary: For adjacent words, first differing chars give edge a->b. If longer word appears before its exact prefix, invalid. Then topo-sort characters.
Word ladder BFS: Vertices are words; edges connect words differing in one char. BFS gives min transformations. Wildcard buckets h*t, *ot reduce neighbor generation.
0-1 BFS: For edge weights 0/1, use deque. Relax 0-edge with push_front, 1-edge with push_back. Distances processed nondecreasingly. O(V+E).
Critical path DAG: Tasks with durations and prerequisites: topo order; earliestFinish[v]=dur[v]+max earliestFinish[pred]. Project time=max EF. Cycle => impossible.
Tree centers / min height roots: Repeatedly remove all leaves, decreasing degrees. Remaining 1 or 2 vertices are centers minimizing maximum distance to all nodes. O(n).
Tree diameter 2 BFS: In a tree: BFS/DFS from any node gives farthest a; BFS/DFS from a gives farthest b. Path a-b is a diameter. O(n).
LCA binary tree: Postorder: if node is p/q return node. Let L,R recurse. If both non-null return node; else return non-null child result. O(n).
LCA in BST: If p,q both < root go left; both > root go right; otherwise root is split point and LCA. Runtime O(h).
Euler trail undirected: Ignoring isolated vertices, graph must be connected. Euler circuit iff all degrees even. Euler trail iff exactly 0 or 2 odd-degree vertices.
2-SAT via SCC: Clause (a OR b) becomes implications !a->b and !b->a. Satisfiable iff no variable x has x and !x in same SCC.
Huffman coding: Repeatedly merge two smallest frequencies. Greedy proof: two least frequent symbols can be deepest siblings in some optimum. O(n log n).
Fractional knapsack: Sort by value/weight decreasing, take greedily, last item possibly fractional. Exchange proof by replacing lower density with higher density. 0/1 differs.
Deadline jobs profit: Unit jobs with deadline/profit: sort profit descending, schedule each in latest free slot <=deadline (DSU for slots). Greedy keeps high profit.
Gas station circle: If total gas<total cost no start. Else scan tank; when tank<0, no station in current segment can start, so next index becomes candidate.
Monotonic stack: Next greater element: keep stack decreasing. While top < x, x is next greater for popped indices. Push x. Each element popped once => O(n).

Extra Exam Trap Micro-Stencils

Growth order: log n < log(n!)=Theta(n log n) < n log n < n2 < n3 < 2n < 3n < n! < nn. Also log^k n=o(n^eps), n^k=o(c^n).
Weird growth: (log n)^(log n)=n^(log log n): super-polynomial, sub-exponential. n^(sqrt n)=e^(sqrt n log n) < c^n but > n^k.
Factorial estimate: Stirling: n!=(Theta(sqrt n))(n/e)^n. For MCQ usually enough: log(n!)=sum log i=Theta(n log n).
Recurrence T(n-1): T(n)=T(n-1)+f(n) => sum f. T(n)=aT(n-1)+1 with a>1 => Theta(a^n).
Recurrence sqrt: T(n)=aT(sqrt n)+f(n). Put m=log n, S(m)=aS(m/2)+f(2^m). Example T(n)=4T(n^(1/4))+log n => S(m)=4S(m/4)+m=Theta(m log m).
Unequal split AB: T(n)=sum a_i T(b_i n)+Theta(n^d). Find p with sum a_i b_i^p=1. p>d => Theta(n^p); p=d => Theta(n^p log n); p<d => Theta(n^d).
AB examples: T(n)=T(n/3)+T(n/2)+n: p<1, answer Theta(n). If alpha^d+beta^d=1 and work n^d, answer Theta(n^d log n).
Strengthening recipe: If substitution gives target + bad lower term, prove stronger upper T(n)<=main-lowerTerm. LowerTerm must create slack under recursive shrink.
Master trap: Master only for aT(n/b)+f(n), fixed a,b. Not for T(n-1), T(sqrt n), unequal splits unless transformed/AB.
Asymptotic proof: To show f=O(g): find c,n0. To show Omega: find c,n0 lower. Theta = both. For sums, dominant term wins if positive.
Stack trace: Push in array order then pop gives reversed order. Queue enqueue then dequeue gives same order. Underflow only if pop/dequeue empty.
Heap indices: 1-index: parent=floor(i/2), left=2i, right=2i+1. 0-index: parent=(i-1)/2, left=2i+1, right=2i+2.
Heap trace: BuildHeap calls Heapify from floor(n/2) downto 1. In max-heap, root is max; heapsort swaps root with end, shrinks heap, so final array ascending.
Stable/in-place MCQ: Insertion stable/in-place. Merge stable/not in-place. Heap in-place/not stable. Random quicksort in-place-ish/not stable, expected n log n, worst n2.
Comparison sort lower: Only comparison sorts have Omega(n log n) lower bound. Counting/radix can beat it using key structure.
Binary lower bound: lower-bound returns first i with A[i]>=x. Inv: answer in [l,r]. If A[mid]>=x then r=mid else l=mid+1.
Binary upper bound: upper-bound returns first i with A[i]>x. Count <=x is upper_bound(x); count <x is lower_bound(x).
Floor/ceil search: floor x = last <=x = upper_bound(x)-1. ceil x = first >=x = lower_bound(x). Check sentinel when index outside array.
BST height MCQ: Any binary tree with n nodes has height at least floor(log2 n). A BST can be height n-1. Search worst O(h).
BST duplicates: State convention. If left < root and right >= root, validation interval should be (lo,key) on left and [key,hi) or adapted on right.
Adjacency matrix/list: List space Theta(V+E), scan neighbors O(deg). Matrix space Theta(V2), edge lookup O(1), scanning neighbors O(V).
Transpose graph: For directed G, build G^T by reversing every edge in adjacency lists. Runtime Theta(V+E). Used in Kosaraju.
Connected components: Undirected: run BFS/DFS from every unvisited vertex; each search outputs one component. Runtime Theta(V+E).
Tree tests: Undirected graph is tree iff connected and E=V-1 iff acyclic and E=V-1 iff connected and acyclic. Forest with c comps has E=V-c.
Bipartite test: BFS color by parity distance. If edge joins same color => not bipartite; else partition is bipartition. Runtime Theta(V+E).
DAG facts: Every nonempty DAG has a source and sink. Repeatedly removing sources gives topo order. Directed cycle means no topo order.
Topo uniqueness: Topo order is unique iff every consecutive pair in the produced order has an edge between them in that direction. Otherwise swap adjacent incomparable vertices.
DFS edge classes: Directed DFS: tree, back(to gray ancestor), forward(to descendant), cross(other). Undirected DFS has only tree/back edges.
SCC condensation DP: Condense SCCs to DAG. To solve reachability/order DP between components, solve on condensation, not original cycles.
MST distinct weights: If all edge weights are distinct, MST is unique. Proof: if two MSTs differ, take lightest edge in symmetric difference and use cut/cycle exchange.
MST equal weights: Equal weights may give multiple MSTs, but Kruskal/Prim still return an MST. Tie-breaking changes tree, not optimum weight.
MST negative weights: Negative edges are fine; cut/cycle properties do not require nonnegative weights. Kruskal still works.
MST vs SPT: MST minimizes total tree weight; shortest-path tree preserves distances from source. Neither implies the other.
MST transform: Add constant c to every edge: MST unchanged because every spanning tree has V-1 edges. Multiply by positive c: unchanged. Negative multiply turns min into max.
Bottleneck tree: Every MST is a minimum-bottleneck spanning tree: its largest edge weight is as small as possible. Converse need not hold.
Bridge/cycle MST: Every bridge is in every spanning tree/MST. A heaviest edge can still be in MST if it is a bridge; cycle property only applies to cycles.
Dijkstra zeros: Zero-weight edges OK. Negative edges not OK because finalized vertex might later improve.
Unreachable vertices: BFS/Dijkstra/BF leave d=INF and parent=NIL. Do not claim negative cycle affects answer unless reachable from source.
BF any negative cycle: To detect any negative cycle anywhere, add super-source s to every vertex with weight 0, then run Bellman-Ford.
Difference constraints: Constraint x_v <= x_u + w(u,v) becomes edge u->v weight w. Feasible iff no negative cycle. Distances from super-source give solution.
Arbitrage: Exchange rates r(u,v). Product cycle >1 iff sum of weights -log r(u,v) on cycle <0. Detect with Bellman-Ford.
Shortest cycle through edge: For directed edge e=(u,v), shortest cycle using e = w(e)+shortest path v->u after removing/allowing e carefully.
Number of shortest paths: During BFS/Dijkstra: if better dist, set count=count[u]; if equal dist, add count[u]. Need process in nondecreasing dist.
Even/odd path: Duplicate states (v,parity). Edge toggles parity. BFS/Dijkstra on product graph; answer dist[(t,desiredParity)].
Mod-state shortest path: For path length/cost mod k, use states (v,r). Edge updates r. Then run BFS/Dijkstra depending on weights.
One special edge/state: If one coupon/free/expensive action allowed, states (v,used). Normal transitions keep used; special transition flips 0->1.
Path reconstruction: Store parent when relaxing/discovering. Reconstruct by following parent from target to source and reversing. For DP store argmin choice.
Residual edge meaning: Forward residual cap = unused capacity. Backward residual cap = current flow that can be canceled. Augmenting may undo earlier choices.
Augment bottleneck: Path push amount = min residual capacity on path. At least one edge becomes saturated, but later back edges can unsaturate it.
Undirected capacity: For undirected edge capacity c, common reduction uses two opposite arcs cap c; opposite flows can be canceled to respect net capacity. For edge-disjoint, use a single cap-1 gadget.
Min edge cut: Set every edge capacity 1. Min s-t cut value = minimum number of edges whose removal separates s,t.
Min vertex cut: Split each nonterminal v into vin->vout cap 1, original edges go out->in with INF. Min cut value = min internal vertices to remove.
Interval DP shape: If answer for interval [i,j] depends on choosing split/root k inside, states are intervals, fill by increasing length. Runtime often O(n3).
Grid DP shape: If moves only right/down/up-diag in acyclic direction, dp[cell]=best over predecessor cells. Fill in topological coordinate order.
Exact-k DP: Add dimension k. dp[i][k]=best using prefix/items with exactly k choices. For at most k, answer max over <=k or carry skip transitions.
Feasibility DP: Boolean dp means possible, transition OR. Optimization dp means best value, transition min/max. Do not mix proof language.
Subset sum: dp[x]=possible sum x. For each item w, update x decreasing: dp[x]|=dp[x-w]. Decreasing prevents using item twice.
Unbounded loop order: Unbounded coin/rod: update amount increasing so same item can be reused. 0/1: capacity decreasing.
Substring vs subsequence: Substring must be contiguous: if chars equal dp[i][j]=dp[i-1][j-1]+1 else 0. Subsequence uses max(skip left, skip right).
DP with sorted choices: Sort by natural endpoint/time/value first. Then predecessor p(i) via binary search or two pointers; recurrence usually take/skip.
Solution reconstruction DP: When taking max/min, store arg choice. After table filled, walk choices backward/recursively to output selected objects.
Pseudo-polynomial: O(nW) is polynomial in numeric value W, not in input bitlength log W. Say pseudo-polynomial if weights are input numbers.
Greedy fail signal: If choices have weights/profits interacting with future capacity, suspect DP not greedy. Example: 0/1 knapsack not ratio-greedy.
Interval scheduling proof: Earliest finishing interval is safe: replace first interval of any optimal solution by it; leaves at least as much room for the rest.
Sweep tie rules: For half-open [s,e), process end before start at same coordinate. For closed intervals, process start before end if touching counts as overlap.
Binary answer bounds: Pick L definitely infeasible or minimal possible, R definitely feasible or maximal possible. State why bounds contain optimum.
Monotone predicate: Minimize maximum: if M feasible, larger M feasible. Maximize minimum: if M feasible, smaller M feasible. Adjust binary search direction.
Random indicators: Linearity of expectation does not require independence. Define indicator for each local event, compute its probability, sum.
Union bound: P(any bad event) <= sum P(bad_i). Combine with expectation: P[X>=1] <= E[X].
Markov: For nonnegative X: P[X>=a] <= E[X]/a. Useful to turn expected bad count into existence/high-prob bound.
Pairwise independence: For expected collision count, pairwise independence/universal hashing is enough; no need full independence over all keys.
Randomized worst case: Randomized quicksort has expected O(n log n), but a particular unlucky pivot sequence still gives O(n2). Expected != worst-case.
Online ratio form: For minimization, ALG(I) <= r OPT(I)+c where c independent of input length. For maximization, usually ALG >= OPT/r - c.
Adversary lower bound: To prove no online alg beats r, adaptively choose continuation after seeing ALG action; compare ALG cost/value with offline OPT on same sequence.
Index convention: Before writing recurrence, define whether intervals are [i,j], [i,j), or prefix first i items. Most off-by-one errors come from this.
Travel pass endpoint: Pass of length L bought on day d covers integer days d..d+L-1. Previous uncovered day must be < d, or next index has day > d+L-1.
INF choice: For reductions, INF can be any number larger than max possible finite objective/cut, e.g. sum of all finite capacities+1.
Certificate habit: For optimization, mention certificate: MST edges total weight, shortest-path distances+parents, maxflow+mincut, DP table+choices.
Runtime habit: State graph size after reduction. If time-expanded/layered graph has T layers, multiply V,E by T before applying BFS/Dijkstra/flow.
Proof ending: Always finish: algorithm returns feasible solution; by lemma/invariant no better solution exists; hence optimal. Then runtime.